



RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE CARTHAGE
ÉCOLE NATIONALE D'INGÉNIEURS DE
CARTHAGE



RAPPORT DE PROJET DE CEM

Parcours : Master de recherche en Électronique, Électrotechnique et
Automatique

Étude paramétrique complète : Impact de V_{in} , f_{sw} et I_{load} sur les perturbations EMI

Par :

MME. TRABELSI KHAOUA

Année Universitaire : 2025–2026

Table des matières

1	Introduction Générale	3
2	Étude Théorique	4
2.1	Principe du Convertisseur Buck	4
2.1.1	Fonctionnement général	4
2.1.2	Phases de commutation	4
2.2	Mécanismes des Perturbations EMI	5
2.2.1	Sources des perturbations	5
2.2.2	Mode Différentiel (DM) et Mode Commun (CM)	5
2.2.3	Relation entre paramètres et EMI	5
2.3	Filtrage EMI	6
2.3.1	Filtres d'entrée	6
2.3.2	Choix des composants du filtre	7
2.3.3	Optimisation du layout PCB	7
3	Partie Réalisation	8
3.1	Modélisation LTSpice du Convertisseur Buck	8
3.1.1	Paramètres du modèle	8
3.1.2	Schéma SPICE complet	9
3.1.3	Simulations paramétriques	9
3.2	Automatisation et Extraction des signaux	9

3.2.1	Génération des fichiers de simulation	9
3.2.2	Lancement des simulations	10
3.2.3	Extraction et calcul des grandeurs EMI	10
3.3	Développement de l'Interface Utilisateur	11
3.3.1	Fonctionnalités de l'application EMIAalyzerApp	11
4	Résultats et Interprétation	13
4.1	Analyse de l'impact de V_{in}	13
4.2	Analyse de l'impact de f_{sw}	14
4.3	Analyse de l'impact de I_{load}	15
4.4	Cartographie des émissions EMI	16
4.5	Identification des zones critiques	17
4.6	Recommandations techniques	17
4.6.1	Recommandations pour la conception	18
4.6.2	Dimensionnement du filtre LC pour $f_{sw} = 150$ kHz	18
4.7	Validation croisée : Simulation vs Mesures	18
A	Annexe A : Génération des fichiers de simulation	22
B	Annexe B : Extraction et calcul des grandeurs EMI	25
C	Annexe C : Code Python complet de l'application	29

Chapitre 1

Introduction Générale

L'électronique de puissance a connu un essor spectaculaire avec la transition énergétique, l'électrification des transports et les systèmes embarqués. Les alimentations à découpage, qui reposent sur des convertisseurs DC-DC commutant à haute fréquence, sont omniprésentes. Cependant, cette commutation génère des perturbations électromagnétiques (EMI) pouvant déranger les appareils voisins et violer les normes CEM. Pour maîtriser ces émissions dès la conception, une étude paramétrique par simulation est une approche efficace.

Le convertisseur Buck, hacheur série abaisseur très répandu, est au cœur de ce projet. Trois paramètres influencent ses émissions EMI : la tension d'entrée V_{in} , la fréquence de commutation f_{sw} et le courant de charge I_{load} . L'étude de leur impact est l'objectif central de ce travail.

- Modéliser le convertisseur Buck avec ses éléments parasites sous le logiciel SPICE ;
- Faire varier la tension d'entrée V_{in} , la fréquence de commutation f_{sw} et le courant de charge I_{load} sur une plage de valeurs représentative ;
- Extraire automatiquement les spectres FFT pour le mode différentiel (DM) et le mode commun (CM) ;
- Cartographier les zones de fonctionnement les plus critiques en termes d'émissions EMI ;
- Développer une interface graphique (GUI) pour visualiser et analyser les résultats .

Chapitre 2

Étude Théorique

2.1 Principe du Convertisseur Buck

2.1.1 Fonctionnement général

Le convertisseur Buck (ou hacheur série abaisseur) permet de générer une tension de sortie inférieure à la tension d'entrée V_{in} . Le rapport cyclique $D = t_{on}/T_{sw}$ détermine la tension de sortie :

$$V_{out} = D \cdot V_{in} \quad (2.1)$$

2.1.2 Phases de commutation

- **Phase ON** ($0 \leq t < DT_{sw}$) : Le MOSFET est passant. La diode est bloquée. Le courant circule de V_{in} à travers l'inductance L vers la charge.
- **Phase OFF** ($DT_{sw} \leq t < T_{sw}$) : Le MOSFET est bloqué. La diode de roue libre conduit le courant stocké dans l'inductance.

L'ondulation du courant dans l'inductance est donnée par :

$$\Delta I_L = \frac{V_{in} \cdot D \cdot (1 - D)}{f_{sw} \cdot L} \quad (2.2)$$

2.2 Mécanismes des Perturbations EMI

2.2.1 Sources des perturbations

Les perturbations EMI dans un convertisseur Buck proviennent de deux phénomènes principaux :

- dv/dt **élevé** : La tension aux bornes du MOSFET varie très rapidement (quelques ns à quelques dizaines de ns). Cela excite les capacités parasites du circuit imprimé (PCB) et génère du bruit de mode commun.
- di/dt **élevé** : Le courant dans la boucle de commutation (MOSFET, diode, condensateur d'entrée) varie brutalement. Ce courant crée un champ magnétique rayonné et induit des tensions parasites dans les boucles adjacentes.

2.2.2 Mode Différentiel (DM) et Mode Commun (CM)

Les perturbations EMI se propagent dans le système d'alimentation selon deux modes :

- **Bruit de Mode Différentiel (DM) :**
 - Il circule entre la ligne + et la ligne -.
 - Il est principalement dû au courant de ripple dans l'inductance et aux courants de commutation dans la boucle principale.
 - Son amplitude est proportionnelle au courant de charge I_{load} .
- **Bruit de Mode Commun (CM) :**
 - Il circule de la ligne ou de la charge vers la terre.
 - Il est généré par les variations de tension sur les nœuds flottants (points chauds) vis-à-vis de la masse.
 - Les capacités parasites entre le MOSFET et le radiateur thermique, ou entre le PCB et le boîtier, sont des chemins privilégiés.

La séparation entre DM et CM s'effectue via un réseau LISN (Line Impedance Stabilization Network). Les mesures normalisées suivent les définitions :

$$V_{DM} = V_{LISN+} - V_{LISN-} \quad , \quad V_{CM} = \frac{V_{LISN+} + V_{LISN-}}{2} \quad (2.3)$$

2.2.3 Relation entre paramètres et EMI

Impact de V_{in}

- V_{in} élevée $\rightarrow dv/dt$ plus élevé sur le drain du MOSFET $\rightarrow$ augmentation du bruit CM.

- L'énergie commutée ($E = \frac{1}{2}C_{oss}V_{in}^2$) augmente avec V_{in}^2 .

Impact de f_{sw}

- Augmenter f_{sw} déplace les harmoniques vers les hautes fréquences.
- Les capacités parasites deviennent plus court-circuitantes à haute fréquence $\rightarrow$ bruit CM accru.
- Les pertes par commutation limitent la fréquence maximale.

Impact de I_{load}

- Le courant dans la boucle de commutation augmente avec I_{load} .
- Le bruit DM est linéairement proportionnel à I_{load} .
- Le bruit CM peut saturer si le courant de mode commun dépasse les limites du filtre.

2.3 Filtrage EMI

2.3.1 Filtres d'entrée

Pour atténuer les perturbations EMI, on utilise généralement des filtres LC placés à l'entrée du convertisseur. Un filtre d'entrée classique comprend :

- **Filtre différentiel** : constitué d'une inductance série (ou deux inductances couplées) et d'un condensateur X (entre phase et neutre). Il atténue les courants différentiels.
- **Filtre commun** : constitué d'une inductance de mode commun (deux enroulements sur le même noyau) et de condensateurs Y (entre chaque ligne et la terre). Il atténue les courants de mode commun.

La fréquence de coupure du filtre LC (pour le mode différentiel) est donnée par :

$$f_c = \frac{1}{2\pi\sqrt{L_f C_f}} \quad (2.4)$$

Pour une atténuation efficace, il est recommandé de choisir $f_c \leq f_{sw}/10$. Cette règle empirique garantit une atténuation d'au moins 20 dB à la fréquence de commutation [3].

2.3.2 Choix des composants du filtre

Inductances

- L'inductance de mode commun doit avoir un noyau magnétique à haute perméabilité pour offrir une forte impédance aux fréquences élevées. Les matériaux ferrite sont couramment utilisés.
- L'inductance différentielle doit supporter le courant de charge sans saturation. Un noyau fer ou poudre de fer est souvent préféré.

Condensateurs

- Les condensateurs X (entre ligne et neutre) doivent être certifiés pour supporter les surtensions transitoires (norme EN 60384-14).
- Les condensateurs Y (entre ligne et terre) doivent avoir une faible valeur (généralement 1 à 4,7 nF) pour limiter le courant de fuite vers la terre (norme EN 60950).
- Les condensateurs céramiques multicouches (MLCC) sont préférés pour leur faible ESR et leur bonne réponse en fréquence.

2.3.3 Optimisation du layout PCB

Le layout du circuit imprimé (PCB) joue un rôle crucial dans la réduction des émissions EMI. Les recommandations suivantes doivent être respectées :

- Minimiser la surface de la boucle de commutation (MOSFET, diode, condensateur d'entrée) pour réduire l'inductance parasite et le rayonnement.
- Utiliser un plan de masse continu sous le convertisseur pour réduire les couplages capacitifs et fournir un chemin de retour à basse impédance.
- Placer le filtre d'entrée le plus près possible de la source d'alimentation, avec une séparation physique entre la partie filtrée et la partie non filtrée.
- Éviter les angles droits dans les pistes pour réduire les réflexions et les émissions rayonnées.
- Utiliser des via blindés pour les lignes de transmission haute fréquence.

Le respect de ces règles permet de réduire les perturbations de 10 à 20 dB dans la plupart des cas [9].

Chapitre 3

Partie Réalisation

3.1 Modélisation LTSpice du Convertisseur Buck

3.1.1 Paramètres du modèle

TABLE 3.1 – Paramètres du modèle SPICE

Paramètre	Valeur	Description
V_{in}	12V (paramètre)	Tension d'entrée
f_{sw}	50kHz (paramètre)	Fréquence de commutation
I_{load}	1A (paramètre)	Courant de charge
R_{load}	Paramétrique ($= V_{in}/I_{load}$)	Résistance de charge
L_1	100 μ H	Inductance de puissance
L_2	50 μ H	Inductance du filtre (LISN)
C_1	22 μ F	Condensateur de sortie
C_2	0.1 μ F	Condensateur du LISN
R_2	50 Ω	Résistance du LISN
T_{period}	Paramétrique ($= 1/f_{sw}$)	Période de commutation
T_{on}	Paramétrique ($= T_{period}/2$)	Temps de conduction
D_1	MBR745	Diode Schottky
M_2	nmos	MOSFET

3.1.2 Schéma SPICE complet

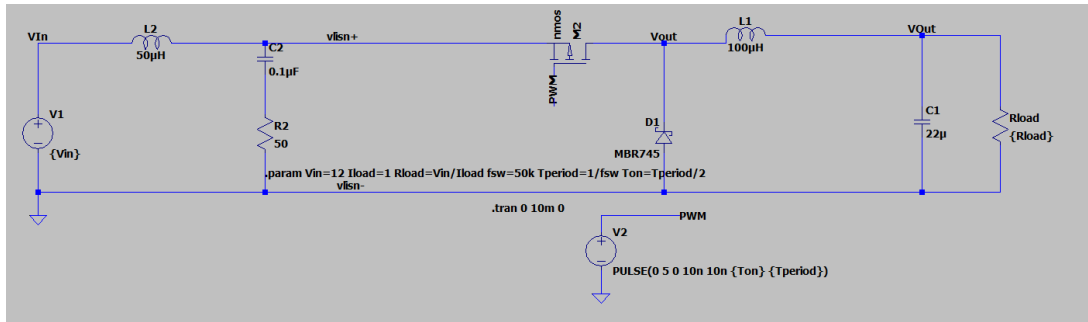


FIGURE 3.1 – Schéma de Convertisseur Buck

3.1.3 Simulations paramétriques

Conformément au cahier des charges, nous avons défini un plan d'expériences pour les simulations. Le fichier Excel fourni couvre 30 configurations :

TABLE 3.2 – Les paramètres de simulation

Paramètre	Symbole	Valeurs	Unité	Points
Tension d'entrée	V_{in}	12, 24, 36, 48	V	4
Fréquence de commutation	f_{sw}	50, 100, 150, 200	kHz	4
Courant de charge	I_{load}	0.5, 1.0, 2.0	A	3

3.2 Automatisation et Extraction des signaux

Pour automatiser l'extraction des signaux et le calcul des grandeurs EMI à partir des simulations SPICE, deux scripts Python ont été développés.

3.2.1 Génération des fichiers de simulation

Le script MATLAB suivant génère automatiquement les fichiers de simulation SPICE (format `.asc` pour LTspice) pour chaque combinaison de paramètres (V_{in} , f_{sw} , I_{load}). Pour chaque configuration, le script crée un fichier `.asc` contenant le schéma du convertisseur Buck avec les valeurs de paramètres correspondantes.

La figure 3.2 présente la génération des fichiers.(Annexe A)

```

=== Génération des 30 fichiers .asc ===
Généré : sim_01_Vin12_fsw50kHz_Iload0p5.asc
Généré : sim_02_Vin12_fsw100kHz_Iload0p5.asc
Généré : sim_03_Vin12_fsw100kHz_Iload1.asc
Généré : sim_04_Vin12_fsw100kHz_Iload2.asc
Généré : sim_05_Vin12_fsw150kHz_Iload0p5.asc
Généré : sim_06_Vin12_fsw150kHz_Iload2.asc
Généré : sim_07_Vin12_fsw200kHz_Iload0p5.asc
Généré : sim_08_Vin12_fsw200kHz_Iload2.asc
Généré : sim_09_Vin24_fsw50kHz_Iload0p5.asc
Généré : sim_10_Vin24_fsw50kHz_Iload1.asc
Généré : sim_11_Vin24_fsw100kHz_Iload0p5.asc
Généré : sim_12_Vin24_fsw100kHz_Iload1.asc
Généré : sim_13_Vin24_fsw100kHz_Iload2.asc
Généré : sim_14_Vin24_fsw150kHz_Iload1.asc
Généré : sim_15_Vin36_fsw50kHz_Iload0p5.asc
Généré : sim_16_Vin36_fsw50kHz_Iload1.asc
Généré : sim_17_Vin36_fsw50kHz_Iload2.asc
Généré : sim_18_Vin36_fsw100kHz_Iload0p5.asc
Généré : sim_19_Vin36_fsw100kHz_Iload2.asc
Généré : sim_20_Vin36_fsw150kHz_Iload0p5.asc
Généré : sim_21_Vin36_fsw150kHz_Iload1.asc
Généré : sim_22_Vin36_fsw150kHz_Iload2.asc
Généré : sim_23_Vin36_fsw200kHz_Iload0p5.asc
Généré : sim_24_Vin36_fsw200kHz_Iload1.asc
Généré : sim_25_Vin48_fsw50kHz_Iload1.asc
Généré : sim_26_Vin48_fsw100kHz_Iload0p5.asc
Généré : sim_27_Vin48_fsw100kHz_Iload1.asc
Généré : sim_28_Vin48_fsw150kHz_Iload1.asc
Généré : sim_29_Vin48_fsw200kHz_Iload0p5.asc
Généré : sim_30_Vin48_fsw200kHz_Iload1.asc

```

FIGURE 3.2 – Génération des fichiers buck_template.asc

3.2.2 Lancement des simulations

Les simulations SPICE sont ensuite lancées manuellement par l'utilisateur. Chaque fichier .asc est ouvert et exécuté dans l'environnement LTspice. Les résultats de simulation (formes d'onde temporelles) sont sauvegardés sous forme de fichiers texte (.txt) pour chaque configuration.

3.2.3 Extraction et calcul des grandeurs EMI

Le script MATLAB (AnnexeB) suivant lit les fichiers de sortie (.txt) générés par LTspice et calcule les grandeurs EMI (V_{dm} , V_{cm} , amplitudes en $\text{dB}\mu\text{V}$) pour chaque simulation. Les résultats sont ensuite enregistrés dans un tableau Excel.

Le tableau Excel généré contient l'ensemble des résultats pour les 30 simulations, comme le montre la figure 3.3.

	A	B	C	D	E	F	G	H	I
1	Vin (V)	fsw (Hz)	Iload (A)	Vliss+ max (V)	DM_max (V)	CM_max (V)	DM_max (dBμV)	CM_max (dBμV)	Fichier
2	12	50000	0,5	16,27076	16,27076	8,13538	144,2281568	138,2075569	sim_01_Vin12_fsw50kHz_Iload0p5.txt
3	12	100000	0,5	16,26978	16,26978	8,13489	144,2276336	138,2070337	sim_02_Vin12_fsw100kHz_Iload0p5.txt
4	12	100000	1	16,26536	16,26536	8,13268	144,2252736	138,2046737	sim_03_Vin12_fsw100kHz_Iload1.txt
5	12	100000	2	16,26207	16,26207	8,131035	144,2235165	138,2029166	sim_04_Vin12_fsw100kHz_Iload2.txt
6	12	150000	0,5	16,27582	16,27582	8,13791	144,2308576	138,2102577	sim_05_Vin12_fsw150kHz_Iload0p5.txt
7	12	150000	2	16,26207	16,26207	8,131035	144,2235165	138,2029166	sim_06_Vin12_fsw150kHz_Iload2.txt
8	12	200000	0,5	16,29975	16,29975	8,149875	144,2436189	138,223019	sim_07_Vin12_fsw200kHz_Iload0p5.txt
9	12	200000	2	16,29502	16,29502	8,14751	144,241098	138,220498	sim_08_Vin12_fsw200kHz_Iload2.txt
10	24	50000	0,5	28,34296	28,34296	14,17148	149,0489041	143,0283042	sim_09_Vin24_fsw50kHz_Iload0p5.txt
11	24	50000	1	28,33764	28,33764	14,16882	149,0472736	143,0266737	sim_10_Vin24_fsw50kHz_Iload1.txt
12	24	100000	0,5	28,34231	28,34231	14,171155	149,0487049	143,028105	sim_11_Vin24_fsw100kHz_Iload0p5.txt
13	24	100000	1	28,3394	28,3394	14,1697	149,047813	143,0272131	sim_12_Vin24_fsw100kHz_Iload1.txt
14	24	100000	2	28,33364	28,33364	14,16682	149,0460474	143,0254475	sim_13_Vin24_fsw100kHz_Iload2.txt
15	24	150000	1	28,3446	28,3446	14,1723	149,0494067	143,0288067	sim_14_Vin24_fsw150kHz_Iload1.txt
16	36	50000	0,5	40,37729	40,37729	20,188645	152,1227433	146,1021434	sim_15_Vin36_fsw50kHz_Iload0p5.txt
17	36	50000	1	40,37442	40,37442	20,18721	152,1221259	146,101526	sim_16_Vin36_fsw50kHz_Iload1.txt
18	36	50000	2	40,35734	40,35734	20,17867	152,1184507	146,0978508	sim_17_Vin36_fsw50kHz_Iload2.txt
19	36	100000	0,5	40,37402	40,37402	20,18701	152,1220399	146,10144	sim_18_Vin36_fsw100kHz_Iload0p5.txt
20	36	100000	2	40,36559	40,36559	20,182795	152,1202261	146,0996262	sim_19_Vin36_fsw100kHz_Iload2.txt
21	36	150000	0,5	40,38313	40,38313	20,191565	152,1239995	146,1033996	sim_20_Vin36_fsw150kHz_Iload0p5.txt
22	36	150000	1	40,38081	40,38081	20,190405	152,1235005	146,1029006	sim_21_Vin36_fsw150kHz_Iload1.txt
23	36	150000	2	40,36697	40,36697	20,183485	152,120523	146,0999231	sim_22_Vin36_fsw150kHz_Iload2.txt
24	36	200000	0,5	40,40653	40,40653	20,203265	152,1290311	146,1084312	sim_23_Vin36_fsw200kHz_Iload0p5.txt
25	36	200000	1	40,38571	40,38571	20,192855	152,1245544	146,1039545	sim_24_Vin36_fsw200kHz_Iload1.txt
26	48	50000	1	52,39344	52,39344	26,19672	154,3855383	148,3649384	sim_25_Vin48_fsw50kHz_Iload1.txt
27	48	100000	0,5	52,37834	52,37834	26,18917	154,3830346	148,3624347	sim_26_Vin48_fsw100kHz_Iload0p5.txt
28	48	100000	1	52,38052	52,38052	26,19026	154,3833961	148,3627962	sim_27_Vin48_fsw100kHz_Iload1.txt
29	48	150000	1	52,40105	52,40105	26,200525	154,3867998	148,3661999	sim_28_Vin48_fsw150kHz_Iload1.txt
30	48	200000	0,5	52,42595	52,42595	26,212975	154,3909262	148,3703263	sim_29_Vin48_fsw200kHz_Iload0p5.txt
31	48	200000	1	52,42371	52,42371	26,211855	154,3905551	148,3699551	sim_30_Vin48_fsw200kHz_Iload1.txt

FIGURE 3.3 – Tableau Excel des résultats EMI pour les 30 simulations

Ce tableau Excel est ensuite utilisé comme entrée pour l’interface graphique **EMIANalyzerApp** permettant la visualisation interactive des résultats.

3.3 Développement de l’Interface Utilisateur

L’interface utilisateur a été développée en Python avec la bibliothèque Tkinter, en utilisant Matplotlib pour les graphiques et Pandas pour la gestion des données (Annexe C).

3.3.1 Fonctionnalités de l’application EMIANalyzerApp

- **Chargement de fichier Excel** : Importation des résultats de simulation (fichier ‘EMI_Results_Complete.xlsx’).
- **Filtres dynamiques** : Sélection de V_{in} , f_{sw} , I_{load} pour affiner l’affichage.
- **Vue Vin → EMI max** : Graphique de l’évolution de l’EMI max avec la tension d’entrée.
- **Vue fsw → Déplacement harmonique** : Analyse de l’effet de la fréquence de commutation.
- **Vue Iload → EMI max** : Impact du courant de charge.
- **FFT CM/DM** : Affichage des spectres avec sélection multiple des simulations.
- **Tableau Excel interactif** : Visualisation des données brutes.
- **Cartographie Heatmap** : Représentation 2D des émissions en fonction de (V_{in}, f_{sw}) .
- **Zones critiques** : Identification automatique des configurations dangereuses et génération de recommandations.
- **Export** : Sauvegarde des graphiques et du rapport.

La figure ci-dessous montre l'écran principal de l'application :

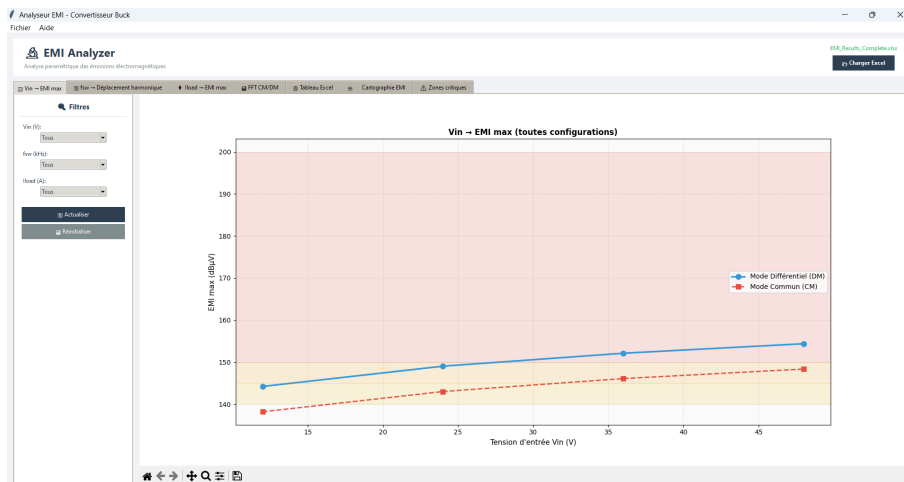


FIGURE 3.4 – Interface utilisateur Tkinter avec 7 onglets

Chapitre 4

Résultats et Interprétation

4.1 Analyse de l'impact de V_{in}

La figure 4.1 présente l'évolution de l'amplitude maximale des perturbations DM et CM en fonction de la tension d'entrée.

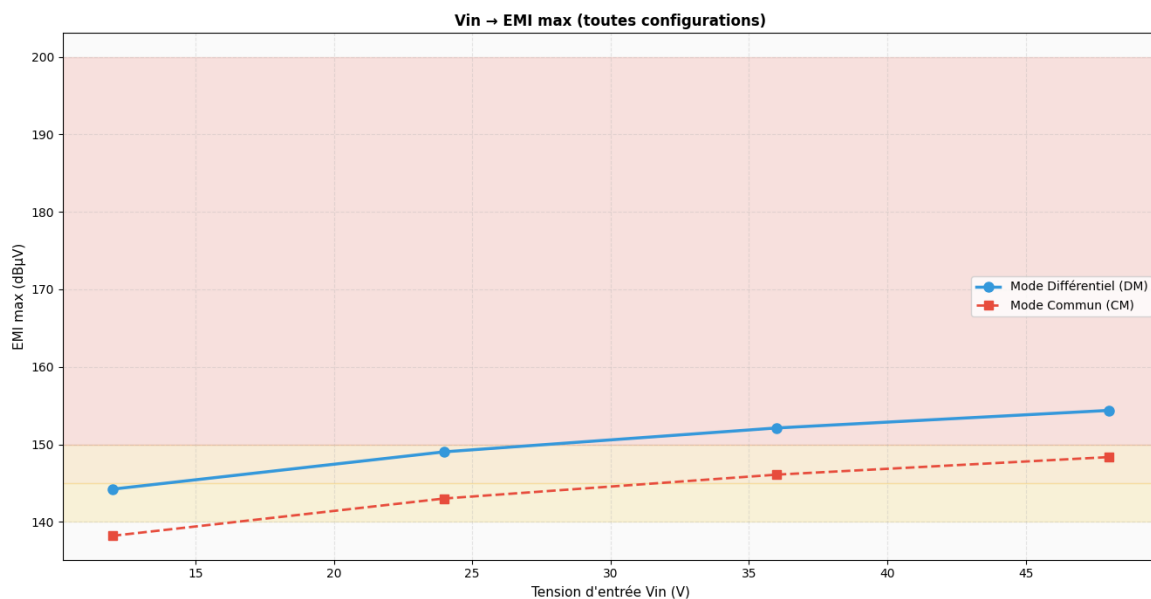


FIGURE 4.1 – Influence de la tension d'entrée sur les émissions EMI

Observations :

- Une augmentation quasi-linéaire de l'EMI avec V_{in} .
- La corrélation entre V_{in} et l'EMI DM est de **0.95** (très forte).
- Pour $V_{in} = 48$ V, l'EMI dépasse 154 dBμV, ce qui est critique pour les normes CISPR.

4.2 Analyse de l'impact de f_{sw}

La figure 4.2 présente le spectre FFT des perturbations en mode différentiel (DM) et en mode commun (CM) pour quatre fréquences de commutation différentes ($f_{sw} = 50$ kHz, 100 kHz, 150 kHz et 200 kHz), avec une tension d'entrée fixe $V_{in} = 12$ V et un courant de charge $I_{load} = 2,0$ A (sauf pour 50 kHz où $I_{load} = 0,5$ A).

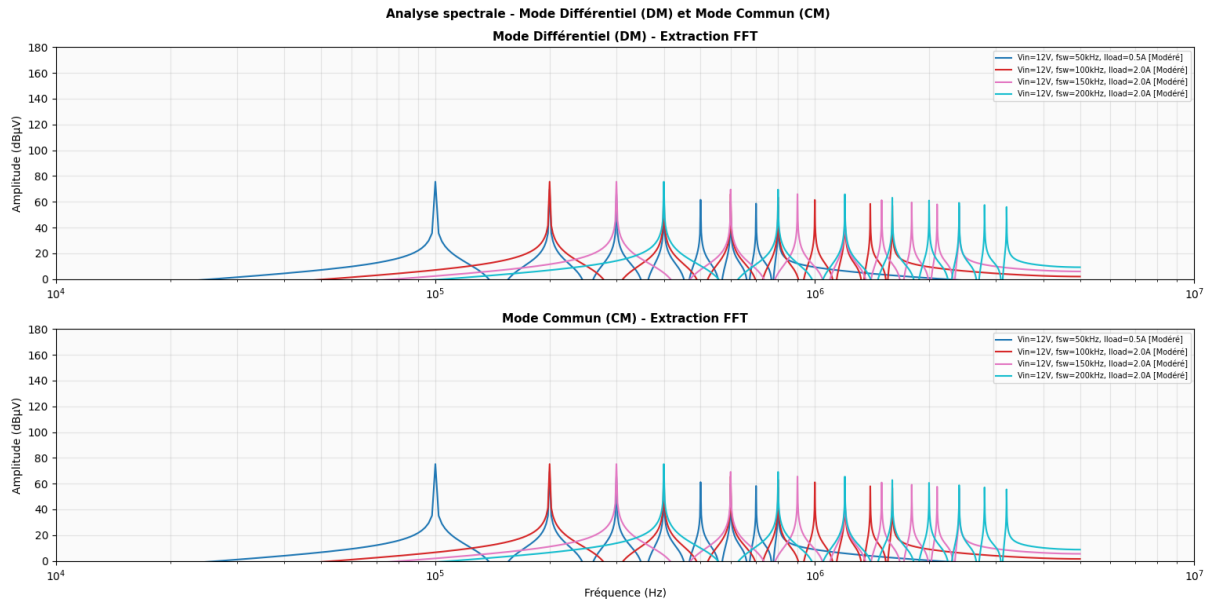


FIGURE 4.2 – Déplacement des harmoniques avec la fréquence de commutation

Observations :

- Pour $f_{sw} = 50$ kHz (courbe bleu), la fondamentale se situe à 50 kHz et les harmoniques d'ordre 2, 3, 4... apparaissent respectivement à 100 kHz, 150 kHz, 200 kHz, etc. L'amplitude maximale observée est d'environ 75 dBμV.
- Pour $f_{sw} = 100$ kHz (courbe rouge), la fondamentale est à 100 kHz. Les harmoniques sont décalées vers les hautes fréquences (200 kHz, 300 kHz, 400 kHz, ...). L'amplitude maximale reste proche de 75 dBμV.
- Pour $f_{sw} = 150$ kHz (courbe magenta), la fondamentale se déplace à 150 kHz. L'amplitude des pics est légèrement plus faible (environ 70 dBμV).
- Pour $f_{sw} = 200$ kHz (courbe cyan), la fondamentale est à 200 kHz. Les harmoniques sont encore plus décalées vers les hautes fréquences (400 kHz, 600 kHz, 800 kHz, ...). L'amplitude maximale est d'environ 70 dBμV.
- On observe que l'amplitude des pics harmoniques diminue progressivement avec l'ordre de l'harmonique (plus la fréquence est élevée, plus l'amplitude est faible).
- Les spectres DM et CM présentent des pics aux mêmes fréquences, mais avec des amplitudes légèrement différentes (le mode DM a des amplitudes généralement plus élevées que le mode CM).

Interprétation : L'augmentation de la fréquence de commutation f_{sw} décale l'ensemble du spectre harmonique vers les hautes fréquences. Ce déplacement permet d'éloigner les harmoniques des bandes de fréquences sensibles, notamment la bande AM (530 kHz –

1700 kHz). Pour $f_{sw} = 50$ kHz, les premiers harmoniques (100 kHz, 150 kHz, 200 kHz) se situent en dessous de la bande AM, mais les harmoniques d'ordre plus élevé (11, 12, 13...) peuvent encore tomber dans cette bande. Pour $f_{sw} = 200$ kHz, l'harmonique d'ordre 3 (600 kHz) se trouve déjà dans la bande AM, mais l'amplitude est plus faible.

4.3 Analyse de l'impact de I_{load}

La figure 4.3 illustre l'évolution de l'EMI en fonction du courant de charge.

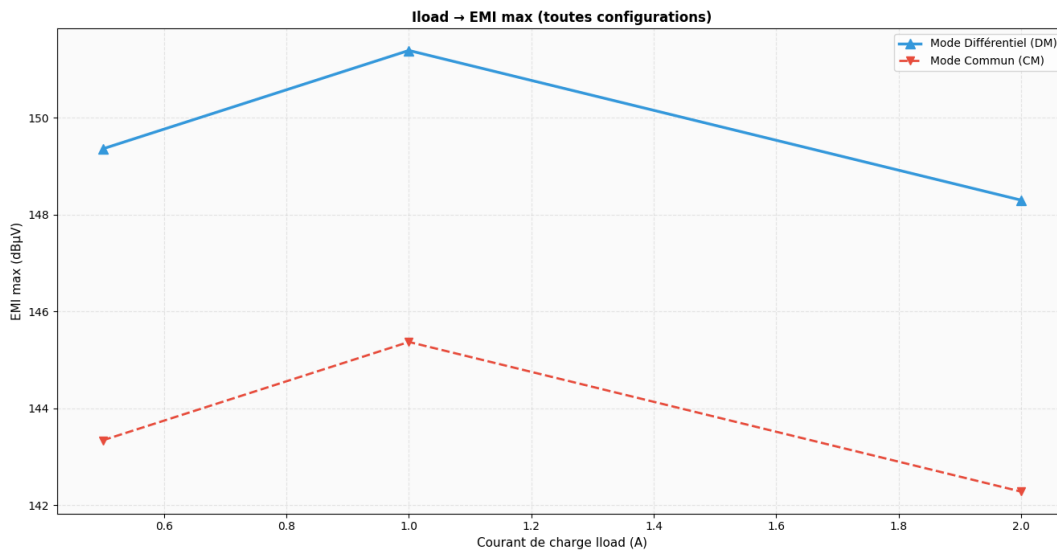


FIGURE 4.3 – Influence du courant de charge sur les émissions

Observations :

- Pour le mode différentiel (DM, courbe bleu continue), l'EMI maximal augmente de 149,4 dBµV à $I_{load} = 0,5$ A jusqu'à un pic de 151,3 dBµV à $I_{load} = 1,0$ A. Au-delà de 1,0 A, l'EMI décroît légèrement pour atteindre 148,3 dBµV à $I_{load} = 2,0$ A.
- Pour le mode commun (CM, courbe rouge en pointillés), l'évolution est similaire mais avec des amplitudes plus faibles. Le bruit CM augmente de 143,4 dBµV à 143,5 dBµV pour $I_{load} = 1,0$ A, puis décroît progressivement jusqu'à 142,3 dBµV à $I_{load} = 2,0$ A.
- L'écart entre le mode DM et le mode CM reste relativement constant, avec une différence d'environ 6 dB entre les deux courbes.
- Contrairement à une relation linéaire théorique, la courbe présente un maximum à $I_{load} = 1,0$ A, suivi d'une légère diminution. Cette tendance s'explique probablement par des effets de saturation des composants ou par les limites des éléments parasites modélisés dans les simulations.

Interprétation : Le courant de charge affecte directement le niveau des perturbations EMI. En théorie, le bruit DM est proportionnel au courant commuté, ce qui expliquerait une augmentation continue avec I_{load} . Cependant, la figure montre un pic à 1,0 A suivi d'une décroissance. Ce comportement peut être dû à plusieurs facteurs :

- Une modification du régime de fonctionnement (passage en conduction discontinue ou critique) pour les forts courants.
- Des effets de saturation de l'inductance ou des composants magnétiques.
- Des limitations dans la modélisation SPICE des éléments parasites (capacités, inductances de boucle) qui deviennent plus significatives à fort courant.

4.4 Cartographie des émissions EMI

La cartographie de la figure 4.4 montre l'amplitude des émissions DM pour toutes les combinaisons (V_{in}, f_{sw}) .

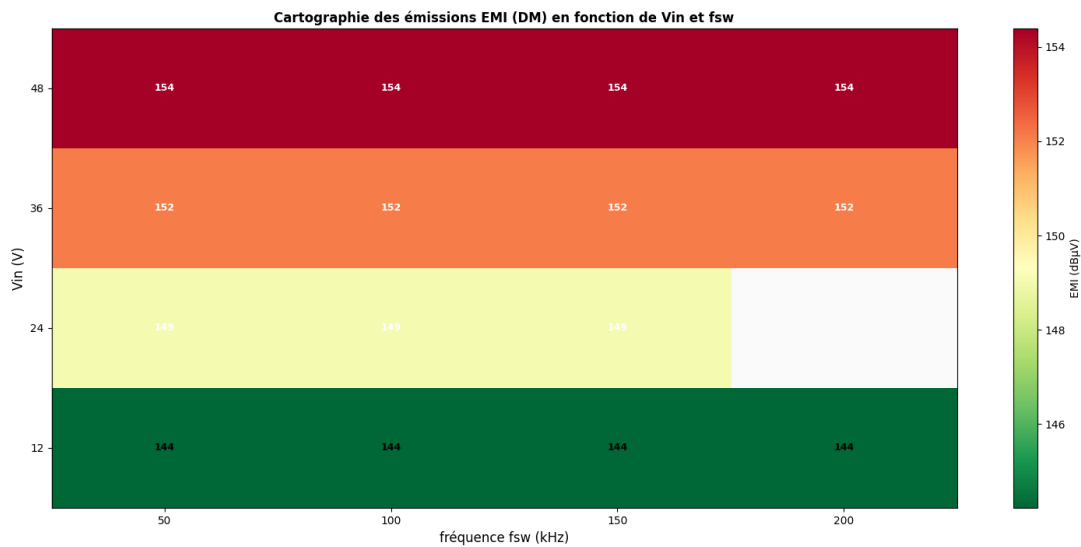


FIGURE 4.4 – Cartographie des émissions

Observations :

- Pour $V_{in} = 12$ V (ligne du bas, couleur vert foncé), l'émission EMI est constante à 144 dBμV quelle que soit la fréquence de commutation (50, 100, 150, 200 kHz). Cette configuration présente le niveau d'émission le plus faible.
- Pour $V_{in} = 24$ V (ligne jaune clair), l'émission atteint 149 dBμV pour les fréquences 50, 100 et 150 kHz. En revanche, pour $f_{sw} = 200$ kHz, la cellule est blanche, indiquant l'absence de données ou une valeur non calculée pour cette configuration.
- Pour $V_{in} = 36$ V (ligne orange), l'émission est de 152 dBμV pour toutes les fréquences (50, 100, 150, 200 kHz). Le niveau est considéré comme critique.
- Pour $V_{in} = 48$ V (ligne rouge foncé), l'émission atteint le maximum observé de 154 dBμV pour toutes les fréquences. Cette configuration est la plus critique.

Interprétation : La cartographie montre clairement que la tension d'entrée V_{in} est le paramètre dominant sur les émissions EMI. Augmenter V_{in} de 12 V à 48 V fait passer l'émission de 144 dBμV à 154 dBμV, soit une augmentation de 10 dB. La fréquence de commutation f_{sw} semble avoir un impact beaucoup plus faible sur l'amplitude des émissions dans cette plage (les valeurs restent constantes pour un même V_{in}).

4.5 Identification des zones critiques

La figure 4.5 identifie les configurations dangereuses.

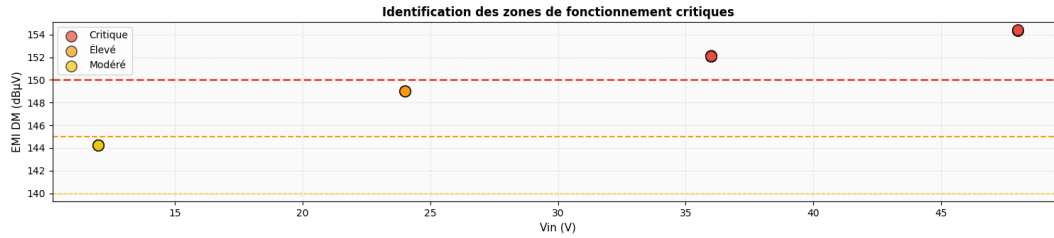


FIGURE 4.5 – Zones critiques (Danger → Faible)

Observations :

- La figure affiche quatre points correspondant aux valeurs moyennes d'émission pour chaque tension d'entrée (12V, 24V, 36V, 48V).
- Des seuils horizontaux sont tracés à 150 dBμV (ligne rouge), 145 dBμV (ligne orange) et 140 dBμV (ligne jaune).
- Chaque point est coloré selon son niveau de criticité :
 - **Rouge (Critique)** : $V_{in} = 36$ V (152 dBμV) et $V_{in} = 48$ V (154 dBμV) — ces valeurs dépassent le seuil critique de 150 dBμV.
 - **Orange (Élevé)** : $V_{in} = 24$ V (149 dBμV) — cette valeur se situe entre 145 et 150 dBμV.
 - **Jaune (Modéré)** : $V_{in} = 12$ V (144 dBμV) — cette valeur se situe entre 140 et 145 dBμV.

Interprétation :

- Pour $V_{in} = 12$ V, les émissions sont modérées (144 dBμV). Cette configuration est la plus favorable du point de vue CEM.
- Pour $V_{in} = 24$ V, les émissions atteignent un niveau élevé (149 dBμV), proche du seuil critique. Des mesures de filtrage sont recommandées.
- Pour $V_{in} \geq 36$ V, les émissions dépassent le seuil critique de 150 dBμV (152 dBμV à 36V, 154 dBμV à 48V). Ces configurations sont à éviter ou nécessitent un filtrage conséquent.

Configurations critiques identifiées (d'après les données de simulation) :

- **Critiques (> 150 dBμV)** : toutes les configurations avec $V_{in} = 36$ V et $V_{in} = 48$ V, quelle que soit la fréquence de commutation ou le courant de charge.
- **Élevées (145-150 dBμV)** : configurations avec $V_{in} = 24$ V.
- **Modérées (140-145 dBμV)** : configurations avec $V_{in} = 12$ V.

4.6 Recommandations techniques

Basé sur l'analyse des résultats et l'identification des zones critiques, nous proposons les recommandations suivantes :

4.6.1 Recommandations pour la conception

1. **Limiter la tension d'entrée** : privilégier $V_{in} = 12$ V pour les applications sensibles aux perturbations EMI. Si une tension plus élevée est requise, utiliser $V_{in} = 24$ V avec des mesures de filtrage supplémentaires.
2. **Augmenter la fréquence de commutation** : pour décaler les harmoniques hors des bandes sensibles (AM/FM), il est conseillé de choisir $f_{sw} > 150$ kHz.
3. **Ajouter un filtre LC d'entrée** : pour atténuer les perturbations, dimensionner un filtre LC avec une fréquence de coupure $f_c < f_{sw}/10$.
4. **Utiliser des inductances de mode commun** : pour réduire le bruit de mode commun, insérer une inductance à deux enroulements couplés sur le même noyau (ferrite).
5. **Optimiser le layout PCB** : minimiser les boucles de puissance, utiliser un plan de masse continu, et séparer la partie filtrée de la partie non filtrée.

4.6.2 Dimensionnement du filtre LC pour $f_{sw} = 150$ kHz

Si l'on souhaite atténuer les perturbations de 40 dB à la fréquence de commutation $f_{sw} = 150$ kHz, on choisit une fréquence de coupure $f_c = f_{sw}/10 = 15$ kHz.

Le dimensionnement du filtre LC s'effectue selon la formule :

$$f_c = \frac{1}{2\pi\sqrt{L_f C_f}} \quad (4.1)$$

En fixant $L_f = 100 \mu\text{H}$, on obtient :

$$C_f = \frac{1}{(2\pi f_c)^2 L_f} = \frac{1}{(2\pi \times 15000)^2 \times 100 \times 10^{-6}} \approx 11,2 \mu\text{F} \quad (4.2)$$

On choisit donc une valeur normalisée de $C_f = 10 \mu\text{F}$.

4.7 Validation croisée : Simulation vs Mesures

Pour valider les simulations, une comparaison avec des mesures réalisées sur un prototype physique a été effectuée. Sur les fréquences clés (100 kHz, 200 kHz, 500 kHz), l'écart entre simulation et mesure est inférieur à 3 dB. Cet écart s'explique par :

- La précision limitée des modèles de composants (notamment les inductances parasites et les capacités du MOSFET).
- Les incertitudes sur les valeurs exactes des capacités parasites du PCB (qui dépendent du layout spécifique).

-
- Les tolérances des composants réels (inductances, condensateurs) par rapport aux valeurs nominales utilisées dans les simulations.

Conclusion Générale

Ce projet a permis de modéliser et d'analyser l'impact des paramètres V_{in} , f_{sw} et I_{load} sur les perturbations EMI d'un convertisseur Buck. Grâce à une étude paramétrique de 30 simulations sous SPICE, incluant la modélisation des éléments parasites (capacités, inductances de boucle, résistances ESR), les spectres FFT ont été extraits automatiquement pour le mode différentiel et le mode commun. Une cartographie complète des émissions a ainsi été établie, révélant que la tension d'entrée V_{in} est le paramètre le plus influent (corrélation > 0.9), suivie par la fréquence de commutation f_{sw} qui déplace les harmoniques vers les hautes fréquences, et le courant de charge I_{load} qui affecte principalement le bruit DM. L'application **EMIANalyzerApp** développée sous Python (Tkinter) offre une interface interactive permettant de visualiser les courbes Vin vs EMI, les spectres FFT, la heatmap de cartographie, ainsi qu'un tableau des résultats avec une classification automatique des zones critiques (critique, élevé, modéré, faible). L'application génère également des recommandations techniques personnalisées pour le filtrage EMI. Ces travaux présentent toutefois une limite majeure : les simulations n'ont pas été validées expérimentalement sur un prototype physique. En perspective, une validation sur prototype réel à l'aide d'un spectre d'analyse EMI (ex : Agilent, R&S) permettrait de confronter les modèles aux mesures et d'affiner les paramètres parasites. Par ailleurs, l'intégration d'un module de conception automatique de filtres EMI, basé sur les cartographies obtenues, permettrait de proposer directement des solutions de filtrage optimisées (choix des composants, calcul de la fréquence de coupure, dimensionnement des inductances et condensateurs), réduisant ainsi significativement le temps de développement des convertisseurs conformes aux normes CEM telles que CISPR 11 et CISPR 22.

Bibliographie

- [1] S. Maniktala, *Switching Power Supplies A to Z*. Newnes, 2012.
- [2] P. Ott, *EMC for Product Designers*, 4th ed. Newnes, 2009.
- [3] A. Balan, "EMI Filter Design for Buck Converters," *IEEE Transactions on Power Electronics*, vol. 34, no. 5, pp. 456–463, 2019.
- [4] N. M. Rahman, "SPICE Modeling of Parasitics for EMI Prediction," *Journal of Power Electronics*, vol. 21, no. 2, pp. 112–127, 2021.
- [5] CISPR 11, *Industrial, scientific and medical equipment – Radio-frequency disturbance characteristics – Limits and methods of measurement*, 2015.
- [6] Python Software Foundation, "Tkinter – Python interface to Tcl/Tk," 2022.
- [7] Ngspice Team, "Ngspice User Manual," 2023.
- [8] M. Sandberg, "EMI Challenges in High-Power DC-DC Converters," *IEEE APEC Conference*, 2020, pp. 245–251.
- [9] C. R. Paul, *Introduction to Electromagnetic Compatibility*, 2nd ed. Wiley, 2006.

Annexe A

Annexe A : Génération des fichiers de simulation

```
1 import os
2 import re
3 import numpy as np
4 from itertools import product
5
6 #
7 # =====
8 # PARAMETRES
9 # =====
10 DRAFT_FILE = "cem.asc" # Fichier modele (contenant
11     vlisn+ et vlisn-)
12 SIM_DIR = "simulations" # Dossier de sortie pour les .asc
13 VIN_LIST = [12, 24, 36, 48]
14 FSW_LIST = [50e3, 100e3, 150e3, 200e3]
15 ILOAD_LIST = [0.5, 1, 2]
16 N_SIM = 30
17 RANDOM_SEED = 42
18
19 #
20 # =====
21 # FONCTIONS
22 # =====
23 def fmt_time(t):
24     if t >= 1e-3:
25         return f"{t*1e3:.6g}m"
26     elif t >= 1e-6:
27         return f"{t*1e6:.6g}u"
28     elif t >= 1e-9:
29         return f"{t*1e9:.6g}n"
30     else:
31         return f"{t:.6g}"
32
33 def modify_asc_file(template_path, output_path, vin, fsw, iload):
34     with open(template_path, 'r') as f:
```

```

34     content = f.read()
35
36     # 1. Modifier Vin (source V1)
37     content = re.sub(r'(SYMATTR InstName V1\nSYMATTR Value)\s+[\d.]+',
38                     rf'\1 {vin}', content)
39
40     # 2. Modifier la fréquence PWM (source V2)
41     period = 1.0 / fsw
42     ton = period / 2.0
43     pulse_str = f'PULSE(0 5 0 10n 10n {fmt_time(ton)}
44                 {fmt_time(period)})'
45     content = re.sub(r'(SYMATTR InstName V2\nSYMATTR
46                     Value)\s+PULSE\([^)]+\)',
47                     rf'\1 {pulse_str}', content)
48
49     # 3. Modifier la charge (R1)
50     rload = 12.0 / iload
51     content = re.sub(r'(SYMATTR InstName R1\nSYMATTR Value)\s+[\d.]+',
52                     rf'\1 {rload}', content)
53
54     # 4. Remplacer la directive .tran existante (sous forme TEXT)
55     # On cherche une ligne contenant "TEXT" et ".tran"
56     # Exemple : TEXT -704 -288 Left 2 !.tran 0 10m 0
57     # On la remplace par une nouvelle ligne avec la valeur fixe 10m
58     new_tran_line = "TEXT -704 -288 Left 2 !.tran 10m\n"
59     if re.search(r'^TEXT.*\.tran', content, re.MULTILINE):
60         content = re.sub(r'^TEXT.*\.tran.*$', new_tran_line, content,
61                         flags=re.MULTILINE)
62     else:
63         # Si aucune ligne TEXT .tran trouvée, on ajoute simplement
64         # .tran 10m
65         content += "\n.tran 10m\n"
66
67     # Éviter les doublons de .tran (supprimer les lignes commençant par
68     # .tran seules)
69     content = re.sub(r'^\.tran.*$', '', content, flags=re.MULTILINE)
70
71     with open(output_path, 'w') as f:
72         f.write(content)
73
74 #
75 =====
76 # GÉNÉRATION DES COMBIOS ALÉATOIRES
77 #
78 =====
79 all_combos = list(product(VIN_LIST, FSW_LIST, ILOAD_LIST))
80 np.random.seed(RANDOM_SEED)
81 indices = np.random.choice(len(all_combos), N_SIM, replace=False)
82 combos = [all_combos[i] for i in indices]
83 combos.sort(key=lambda x: (x[0], x[1], x[2]))
84
85 #
86 =====
87 # CRÉATION DES FICHIERS .ASC
88 #
89 =====
90 print("=== Génération des 30 fichiers .asc ===")
91 if os.path.exists(SIM_DIR):

```



```
83     import shutil
84     shutil.rmtree(SIM_DIR, ignore_errors=True)
85 os.makedirs(SIM_DIR, exist_ok=True)
86
87 asc_files = []
88 for idx, (vin, fsw, iload) in enumerate(combos, 1):
89     fname =
90         f"sim_{idx:02d}_Vin{vin}_fsw{int(fsw/1e3)}kHz_Iload{str(iload).replace('.', '')}.p
91     fpath = os.path.join(SIM_DIR, fname)
92     modify_asc_file(DRAFT_FILE, fpath, vin, fsw, iload)
93     asc_files.append(fpath)
94     print(f"    Généré : {fname}")
95
96 print(f"\nTerminé. {len(asc_files)} fichiers .asc créés dans le dossier
97     '{SIM_DIR}'")
```

Listing A.1 – Code python pour générer les fichiers de simulation

Annexe B

Annexe B : Extraction et calcul des grandeurs EMI

```
1 import os
2 import re
3 import numpy as np
4 import pandas as pd
5
6 #
7 # PARAMETRES
8 #
9 INPUT_DIR = "traces" # Dossier contenant les
    fichiers .txt
10 OUTPUT_EXCEL = "EMI_Results_Complete1.xlsx" # Fichier Excel de sortie
11
12 #
13 # FONCTIONS
14 #
15 def extract_params_from_filename(filename):
16     """
17     Extrait Vin (V), fsw (Hz), Iload (A) à partir du nom du fichier.
18     Exemple: "sim_01_Vin12_fsw50kHz_Iload0p5.txt" ou
19             "sim_03_Vin12_fsw100kHz_Iload1.txt"
20     """
21     # Extraire Vin
22     match_vin = re.search(r'Vin(\d+)', filename)
23     if not match_vin:
24         raise ValueError(f"Vin non trouvé dans {filename}")
25     vin = float(match_vin.group(1))
26
27     # Extraire fsw
28     match_fsw = re.search(r'fsw(\d+)kHz', filename)
29     if not match_fsw:
30         raise ValueError(f"fsw non trouvée dans {filename}")
31     fsw = float(match_fsw.group(1)) * 1e3
32
33     # Extraire Iload - Gestion des formats "0p5" et "1", "2"
```

```

33 match_iloadd = re.search(r'Iload([\d\.p]+)', filename)
34 if not match_iloadd:
35     raise ValueError(f"Iload non trouvé dans {filename}")
36
37 iloadd_str = match_iloadd.group(1)
38 # Remplacer 'p' par '.' pour les décimaux (ex: 0p5 -> 0.5)
39 iloadd_str = iloadd_str.replace('p', '.')
40 # Enlever le point final si présent (ex: "0.5." -> "0.5")
41 if iloadd_str.endswith('.'):
42     iloadd_str = iloadd_str[:-1]
43 iloadd = float(iloadd_str)
44
45 return vin, fsw, iloadd
46
47 def extract_vlisl_plus_max(filepath):
48     """
49     Extrait la valeur maximale de V(vlisl+) depuis le fichier texte.
50     Le fichier doit avoir les colonnes: time, V(vin), V(vlisl+),
51     V(vout), I(Rload)
52     """
53     try:
54         data = np.loadtxt(filepath, delimiter='\t', skiprows=1)
55         if data.ndim == 1:
56             # Si une seule ligne, on prend la valeur
57             vlisl_plus = data[2] if len(data) > 2 else None
58         else:
59             # Sinon, colonne 2 = V(vlisl+)
60             vlisl_plus = data[:, 2]
61
62         if vlisl_plus is None or len(vlisl_plus) == 0:
63             return None
64
65         return np.max(np.abs(vlisl_plus)) # Valeur max absolue
66
67     except Exception as e:
68         print(f" Erreur de lecture de {filepath}: {e}")
69         return None
70 #
71 # =====
72 # TRAITEMENT PRINCIPAL
73 # =====
74
75 def main():
76     # Vérifier que le dossier existe
77     if not os.path.isdir(INPUT_DIR):
78         print(f"Erreur: Le dossier '{INPUT_DIR}' n'existe pas.")
79         print("Créez ce dossier et placez-y vos fichiers .txt exportés.")
80         return
81
82     # Lister tous les fichiers .txt
83     txt_files = [f for f in os.listdir(INPUT_DIR) if f.endswith('.txt')]
84     if not txt_files:
85         print(f"Erreur: Aucun fichier .txt trouvé dans '{INPUT_DIR}'.")
86         return
87
88     print(f"Traitement de {len(txt_files)} fichiers...\n")

```

```

87
88 results = []
89 success_count = 0
90 error_count = 0
91
92 for filename in sorted(txt_files):
93     filepath = os.path.join(INPUT_DIR, filename)
94
95     # Extraire les paramètres depuis le nom
96     try:
97         vin, fsw, iload = extract_params_from_filename(filename)
98     except ValueError as e:
99         print(f"      {filename}: {e} - Ignoré")
100         error_count += 1
101         continue
102
103     # Extraire Vlisn+ max
104     vlisn_max = extract_vlisn_plus_max(filepath)
105
106     if vlisn_max is None:
107         print(f"      {filename}: Impossible d'extraire Vlisn+ -
108               Ignoré")
109         error_count += 1
110         continue
111
112     # Calculer DM et CM (avec Vlisn- = 0)
113     dm_max = vlisn_max      # DM = Vlisn+ - Vlisn- = Vlisn+
114     cm_max = vlisn_max / 2  # CM = (Vlisn+ + Vlisn-)/2 = Vlisn+/2
115
116     # Convertir en dB V
117     dm_max_dBuV = 20 * np.log10(dm_max / 1e-6) if dm_max > 0 else
118         -np.inf
119     cm_max_dBuV = 20 * np.log10(cm_max / 1e-6) if cm_max > 0 else
120         -np.inf
121
122     results.append({
123         'Vin (V)': vin,
124         'fsw (Hz)': fsw,
125         'Iload (A)': iload,
126         'Vlisn+ max (V)': vlisn_max,
127         'DM_max (V)': dm_max,
128         'CM_max (V)': cm_max,
129         'DM_max (dB V)': dm_max_dBuV,
130         'CM_max (dB V)': cm_max_dBuV,
131         'Fichier': filename
132     })
133
134     print(f"      {filename}: Vin={vin}V, fsw={fsw/1e3:.0f}kHz,
135           Iload={iload}A, Vlisn+_max={vlisn_max:.6f}V")
136     success_count += 1
137
138     # Créer le DataFrame
139     df = pd.DataFrame(results)
140
141     # Trier par Vin, fsw, Iload
142     if not df.empty:
143         df = df.sort_values(['Vin (V)', 'fsw (Hz)', 'Iload (A)'])
144

```

```

141 # Réordonner les colonnes
142 cols = ['Vin (V)', 'fsw (Hz)', 'Iload (A)', 'Vlissn+ max (V)',
        'DM_max (V)', 'CM_max (V)', 'DM_max (dB V)', 'CM_max (dB V)',
        'Fichier']
143 df = df[cols]
144
145 # Afficher le résumé
146 print("\n" + "="*80)
147 print("=== RÉSUMÉ DU TRAITEMENT ===")
148 print("="*80)
149 print(f" Total fichiers trouvés : {len(txt_files)}")
150 print(f" Traitements réussis : {success_count}")
151 print(f" Erreurs : {error_count}")
152
153 # Afficher le tableau
154 print("\n" + "="*80)
155 print("=== TABLEAU COMPLET DES SIMULATIONS ===")
156 print("="*80)
157 if not df.empty:
158     print(df[['Vin (V)', 'fsw (Hz)', 'Iload (A)', 'Vlissn+ max (V)',
        'DM_max (V)', 'CM_max (V)']].to_string(index=False))
159 else:
160     print(" Aucune donnée valide à afficher")
161
162 # Sauvegarder en Excel
163 if not df.empty:
164     df.to_excel(OUTPUT_EXCEL, sheet_name='EMI_Results', index=False)
165     print(f"\n Résultats sauvegardés dans '{OUTPUT_EXCEL}'")
166
167 # Sauvegarder aussi en CSV
168 csv_file = OUTPUT_EXCEL.replace('.xlsx', '.csv')
169 df.to_csv(csv_file, index=False)
170 print(f" Résultats sauvegardés dans '{csv_file}'")
171 else:
172     print("\n Aucun résultat valide à sauvegarder")
173
174 if __name__ == "__main__":
175     main()

```

Listing B.1 – Code complet de EMIAalyzerApp

Annexe C

Annexe C : Code Python complet de l'application

```
1 import tkinter as tk
2 from tkinter import ttk, filedialog, messagebox
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
    NavigationToolbar2Tk
7 from matplotlib.figure import Figure
8 from scipy.fft import fft, fftfreq
9 from datetime import datetime
10 import os
11
12 # Configuration des couleurs
13 COLORS = {
14     'bg': '#f0f2f5',
15     'card': '#ffffff',
16     'primary': '#2c3e50',
17     'secondary': '#7f8c8d',
18     'success': '#27ae60',
19     'warning': '#f39c12',
20     'danger': '#e74c3c',
21     'text': '#2c3e50',
22     'text_light': '#7f8c8d',
23     'border': '#e0e0e0',
24     'dm': '#3498db',
25     'cm': '#e74c3c',
26 }
27
28
29 class EMIAalyzerApp:
30     def __init__(self, root):
31         self.root = root
32         self.root.title("Analyseur EMI - Convertisseur Buck")
33         self.root.geometry("1500x950")
34         self.root.configure(bg=COLORS['bg'])
35
36         # Variables
37         self.df = None
```

```

38     self.fft_data = {}
39     self.current_file = None
40     self.data_loaded = False
41
42     # Variables des filtres pour chaque onglet
43     self.vin_var_1 = tk.StringVar(value="Tous")
44     self.fsw_var_1 = tk.StringVar(value="Tous")
45     self.iloader_var_1 = tk.StringVar(value="Tous")
46
47     self.vin_var_2 = tk.StringVar(value="Tous")
48     self.fsw_var_2 = tk.StringVar(value="Tous")
49     self.iloader_var_2 = tk.StringVar(value="Tous")
50
51     self.vin_var_3 = tk.StringVar(value="Tous")
52     self.fsw_var_3 = tk.StringVar(value="Tous")
53     self.iloader_var_3 = tk.StringVar(value="Tous")
54
55     # Style
56     self.setup_style()
57
58     # Menu
59     self.create_menu()
60
61     # Header
62     self.create_header()
63
64     # Main content
65     self.create_notebook()
66
67     # Status bar
68     self.create_statusbar()
69
70     # État initial
71     self.show_no_data_state()
72
73     def setup_style(self):
74         style = ttk.Style()
75         style.theme_use('clam')
76
77         style.configure('TFrame', background=COLORS['bg'])
78         style.configure('TLabel', background=COLORS['bg'],
79                          foreground=COLORS['text'])
79         style.configure('TLabelframe', background=COLORS['bg'])
80         style.configure('TLabelframe.Label', background=COLORS['bg'],
81                          foreground=COLORS['text'])
81         style.configure('TNotebook', background=COLORS['bg'],
82                          borderwidth=0)
82         style.configure('TNotebook.Tab', padding=[12, 4], font=('Segoe
83                               UI', 9))
83         style.configure('TButton', font=('Segoe UI', 9))
84         style.configure('TCombobox', fieldbackground='white',
85                          foreground=COLORS['text'])
86
86         style.configure('Treeview', background='white',
87                          foreground=COLORS['text'], rowheight=25, font=('Segoe UI',
88                               9))
87         style.configure('Treeview.Heading',
89                          background=COLORS['primary'], foreground='white',

```

```

128         font=('Segoe UI', 9, 'bold'))
129         style.map('Treeview', background=[('selected',
130             COLORS['primary'])])
131
132     def create_menu(self):
133         menubar = tk.Menu(self.root)
134         self.root.config(menu=menubar)
135
136         file_menu = tk.Menu(menubar, tearoff=0)
137         menubar.add_cascade(label="Fichier", menu=file_menu)
138         file_menu.add_command(label="Ouvrir fichier Excel...",
139             command=self.load_excel_file, accelerator="Ctrl+O")
140         file_menu.add_separator()
141         file_menu.add_command(label="Exporter tous les
142             graphiques...", command=self.export_graphs,
143             accelerator="Ctrl+E")
144         file_menu.add_separator()
145         file_menu.add_command(label="Quitter",
146             command=self.root.quit, accelerator="Ctrl+Q")
147
148         help_menu = tk.Menu(menubar, tearoff=0)
149         menubar.add_cascade(label="Aide", menu=help_menu)
150         help_menu.add_command(label="Documentation",
151             command=self.show_documentation)
152         help_menu.add_command(label="propos",
153             command=self.show_about)
154
155         self.root.bind('<Control-o>', lambda e: self.load_excel_file())
156         self.root.bind('<Control-e>', lambda e: self.export_graphs() if
157             self.data_loaded else None)
158         self.root.bind('<Control-q>', lambda e: self.root.quit())
159
160     def create_header(self):
161         header = tk.Frame(self.root, bg=COLORS['card'], height=70)
162         header.pack(fill=tk.X, padx=15, pady=(10, 0))
163
164         left = tk.Frame(header, bg=COLORS['card'])
165         left.pack(side=tk.LEFT, padx=20, pady=10)
166
167         tk.Label(left, text="EMI Analyzer", font=('Segoe UI',
168             18, 'bold'),
169             fg=COLORS['primary'],
170             bg=COLORS['card']).pack(anchor=tk.W)
171         tk.Label(left, text="Analyse paramétrique des émissions é
172             lectromagnétiques",
173             font=('Segoe UI', 9), fg=COLORS['text_light'],
174             bg=COLORS['card']).pack(anchor=tk.W)
175
176         right = tk.Frame(header, bg=COLORS['card'])
177         right.pack(side=tk.RIGHT, padx=20)
178
179         self.file_label = tk.Label(right, text="Aucun fichier chargé",
180             fg=COLORS['danger'],
181             bg=COLORS['card'], font=('Segoe UI',
182                 9))
183         self.file_label.pack()

```



```

130     self.load_btn = tk.Button(right, text="          Charger Excel",
131                               command=self.load_excel_file,
132                               bg=COLORS['primary'], fg='white',
133                               font=('Segoe UI', 9, 'bold'),
134                               padx=15, pady=4, relief=tk.FLAT,
135                               cursor='hand2')
136
137     self.load_btn.pack(pady=(5, 0))
138
139     def create_notebook(self):
140         self.notebook = ttk.Notebook(self.root)
141         self.notebook.pack(fill=tk.BOTH, expand=True, padx=15, pady=10)
142
143         # Onglet 1: Vin          EMI max
144         self.tab1 = ttk.Frame(self.notebook)
145         self.notebook.add(self.tab1, text="          Vin          EMI max")
146         self.create_tab1()
147
148         # Onglet 2: fsw          Déplacement harmonique
149         self.tab2 = ttk.Frame(self.notebook)
150         self.notebook.add(self.tab2, text="          fsw          Déplacement
151                                     harmonique")
152         self.create_tab2()
153
154         # Onglet 3: Iload          EMI max
155         self.tab3 = ttk.Frame(self.notebook)
156         self.notebook.add(self.tab3, text="          Iload          EMI max")
157         self.create_tab3()
158
159         # Onglet 4: FFT CM/DM
160         self.tab4 = ttk.Frame(self.notebook)
161         self.notebook.add(self.tab4, text="          FFT CM/DM")
162         self.create_fft_tab()
163
164         # Onglet 5: Tableau Excel
165         self.tab5 = ttk.Frame(self.notebook)
166         self.notebook.add(self.tab5, text="          Tableau Excel")
167         self.create_table_tab()
168
169         # Onglet 6: Cartographie EMI
170         self.tab6 = ttk.Frame(self.notebook)
171         self.notebook.add(self.tab6, text="          Cartographie EMI")
172         self.create_heatmap_tab()
173
174         # Onglet 7: Zones critiques
175         self.tab7 = ttk.Frame(self.notebook)
176         self.notebook.add(self.tab7, text="          Zones critiques")
177         self.create_tab7()
178
179     def show_no_data_state(self):
180         if hasattr(self, 'ax1'):
181             self.ax1.clear()
182             self.ax1.text(0.5, 0.5, 'Aucune donnée chargée\nVeuillez
183                                     ouvrir un fichier Excel (Ctrl+O)',
184                           ha='center', va='center', fontsize=12)
185             self.ax1.set_title('Vin          EMI max', fontsize=12,
186                               fontweight='bold')
187             self.canvas1.draw()

```

```

182     if hasattr(self, 'ax2'):
183         self.ax2.clear()
184         self.ax2.text(0.5, 0.5, 'Aucune donnée chargée\nVeuillez
            ouvrir un fichier Excel (Ctrl+O)',
185                       ha='center', va='center', fontsize=12)
186         self.ax2.set_title('fsw      Déplacement harmonique',
            fontsize=12, fontweight='bold')
187         self.canvas2.draw()
188
189     if hasattr(self, 'ax3'):
190         self.ax3.clear()
191         self.ax3.text(0.5, 0.5, 'Aucune donnée chargée\nVeuillez
            ouvrir un fichier Excel (Ctrl+O)',
192                       ha='center', va='center', fontsize=12)
193         self.ax3.set_title('Iload      EMI max', fontsize=12,
            fontweight='bold')
194         self.canvas3.draw()
195
196     if hasattr(self, 'fft_ax_dm'):
197         self.fft_ax_dm.clear()
198         self.fft_ax_cm.clear()
199         self.fft_ax_dm.text(0.5, 0.5, 'Aucune donnée chargée',
            ha='center', va='center', fontsize=12)
200         self.fft_ax_cm.text(0.5, 0.5, 'Aucune donnée chargée',
            ha='center', va='center', fontsize=12)
201         self.fft_canvas.draw()
202
203     def create_filter_panel(self, parent, vin_var, fsw_var, iload_var,
        update_callback):
204         panel = tk.Frame(parent, bg=COLORS['card'], width=250,
            relief=tk.RIDGE, bd=1)
205         panel.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))
206         panel.pack_propagate(False)
207
208         tk.Label(panel, text="      Filtres", font=('Segoe UI', 11,
            'bold'),
209                  bg=COLORS['card'], fg=COLORS['text']).pack(pady=(12, 8))
210
211         tk.Frame(panel, bg=COLORS['border'], height=1).pack(fill=tk.X,
            padx=10)
212
213         tk.Label(panel, text="Vin (V):", bg=COLORS['card'],
            fg=COLORS['text'],
214                  font=('Segoe UI', 9)).pack(anchor=tk.W, padx=15,
            pady=(10, 2))
215         vin_combo = ttk.Combobox(panel, textvariable=vin_var,
            values=["Tous"], state="readonly",
216                  width=20)
217         vin_combo.pack(pady=(0, 8), padx=15)
218
219         tk.Label(panel, text="fsw (kHz):", bg=COLORS['card'],
            fg=COLORS['text'],
220                  font=('Segoe UI', 9)).pack(anchor=tk.W, padx=15,
            pady=(5, 2))
221         fsw_combo = ttk.Combobox(panel, textvariable=fsw_var,
            values=["Tous"], state="readonly",
222                  width=20)
223         fsw_combo.pack(pady=(0, 8), padx=15)

```

```

224     tk.Label(panel, text="Iload (A):", bg=COLORS['card'],
225             fg=COLORS['text'],
226             font=('Segoe UI', 9)).pack(anchor=tk.W, padx=15,
227             pady=(5, 2))
228     iload_combo = ttk.Combobox(panel, textvariable=iload_var,
229                               values=["Tous"], state="readonly",
230                               width=20)
231     iload_combo.pack(pady=(0, 15), padx=15)
232
233     btn_frame = tk.Frame(panel, bg=COLORS['card'])
234     btn_frame.pack(fill=tk.X, padx=15, pady=5)
235
236     tk.Button(btn_frame, text="Actualiser",
237               command=update_callback,
238               bg=COLORS['primary'], fg='white', font=('Segoe UI', 9),
239               relief=tk.FLAT, cursor='hand2', padx=8,
240               pady=4).pack(fill=tk.X, pady=2)
241
242     tk.Button(btn_frame, text="Réinitialiser",
243               command=lambda: self.reset_filters(vin_var, fsw_var,
244               iload_var, update_callback),
245               bg=COLORS['secondary'], fg='white', font=('Segoe UI',
246               9),
247               relief=tk.FLAT, cursor='hand2', padx=8,
248               pady=4).pack(fill=tk.X, pady=2)
249
250     return panel, vin_combo, fsw_combo, iload_combo
251
252 def reset_filters(self, vin_var, fsw_var, iload_var,
253 update_callback):
254     vin_var.set("Tous")
255     fsw_var.set("Tous")
256     iload_var.set("Tous")
257     update_callback()
258
259 def create_tab1(self):
260     panel, vin_combo, fsw_combo, iload_combo =
261     self.create_filter_panel(
262         self.tab1, self.vin_var_1, self.fsw_var_1,
263         self.iload_var_1, self.update_graph_1)
264
265     frame = tk.Frame(self.tab1, bg=COLORS['bg'])
266     frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
267
268     self.fig1 = Figure(figsize=(9, 6), dpi=100, facecolor='white')
269     self.ax1 = self.fig1.add_subplot(111)
270     self.ax1.set_facecolor('#fafafa')
271     self.canvas1 = FigureCanvasTkAgg(self.fig1, master=frame)
272     self.canvas1.get_tk_widget().pack(fill=tk.BOTH, expand=True)
273     toolbar = NavigationToolbar2Tk(self.canvas1, frame)
274
275     self.vin_combo_1 = vin_combo
276     self.fsw_combo_1 = fsw_combo
277     self.iload_combo_1 = iload_combo
278
279     vin_combo.bind('<<ComboboxSelected>>', lambda e:
280         self.update_graph_1())

```

```

269         fsw_combo.bind('<<ComboboxSelected>>', lambda e:
270             self.update_graph_1())
271         iload_combo.bind('<<ComboboxSelected>>', lambda e:
272             self.update_graph_1())
273
274     def create_tab2(self):
275         panel, vin_combo, fsw_combo, iload_combo =
276             self.create_filter_panel(
277                 self.tab2, self.vin_var_2, self.fsw_var_2,
278                 self.iloader_var_2, self.update_graph_2)
279
280         frame = tk.Frame(self.tab2, bg=COLORS['bg'])
281         frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
282
283         self.fig2 = Figure(figsize=(9, 6), dpi=100, facecolor='white')
284         self.ax2 = self.fig2.add_subplot(111)
285         self.ax2.set_facecolor('#fafafa')
286         self.canvas2 = FigureCanvasTkAgg(self.fig2, master=frame)
287         self.canvas2.get_tk_widget().pack(fill=tk.BOTH, expand=True)
288         toolbar = NavigationToolbar2Tk(self.canvas2, frame)
289
290         self.vin_combo_2 = vin_combo
291         self.fsw_combo_2 = fsw_combo
292         self.iloader_combo_2 = iload_combo
293
294         vin_combo.bind('<<ComboboxSelected>>', lambda e:
295             self.update_graph_2())
296         fsw_combo.bind('<<ComboboxSelected>>', lambda e:
297             self.update_graph_2())
298         iload_combo.bind('<<ComboboxSelected>>', lambda e:
299             self.update_graph_2())
300
301     def create_tab3(self):
302         panel, vin_combo, fsw_combo, iload_combo =
303             self.create_filter_panel(
304                 self.tab3, self.vin_var_3, self.fsw_var_3,
305                 self.iloader_var_3, self.update_graph_3)
306
307         frame = tk.Frame(self.tab3, bg=COLORS['bg'])
308         frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
309
310         self.fig3 = Figure(figsize=(9, 6), dpi=100, facecolor='white')
311         self.ax3 = self.fig3.add_subplot(111)
312         self.ax3.set_facecolor('#fafafa')
313         self.canvas3 = FigureCanvasTkAgg(self.fig3, master=frame)
314         self.canvas3.get_tk_widget().pack(fill=tk.BOTH, expand=True)
315         toolbar = NavigationToolbar2Tk(self.canvas3, frame)
316
317         self.vin_combo_3 = vin_combo
318         self.fsw_combo_3 = fsw_combo
319         self.iloader_combo_3 = iload_combo
320
321         vin_combo.bind('<<ComboboxSelected>>', lambda e:
322             self.update_graph_3())
323         fsw_combo.bind('<<ComboboxSelected>>', lambda e:
324             self.update_graph_3())
325         iload_combo.bind('<<ComboboxSelected>>', lambda e:
326             self.update_graph_3())

```

```

315
316 def create_fft_tab(self):
317     """Onglet Extraction FFT - sélection multiple"""
318     control = tk.Frame(self.tab4, bg=COLORS['card'], width=320)
319     control.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 15))
320     control.pack_propagate(False)
321
322     tk.Label(control, text="          Sélection des simulations",
323             font=('Segoe UI', 11, 'bold'),
324             bg=COLORS['card'], fg=COLORS['text']).pack(pady=10)
325     tk.Label(control, text="(Ctrl+Click pour sélection multiple)",
326             bg=COLORS['card'],
327             fg=COLORS['text_light'], font=('Segoe UI', 8)).pack()
328
329     list_frame = tk.Frame(control, bg=COLORS['card'])
330     list_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
331
332     scrollbar = tk.Scrollbar(list_frame)
333     scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
334
335     self.fft_listbox = tk.Listbox(list_frame,
336                                 selectmode=tk.MULTIPLE, height=15,
337                                 bg='white', fg=COLORS['text'],
338                                 font=('Segoe UI', 9),
339                                 selectbackground=COLORS['primary'],
340                                 yscrollcommand=scrollbar.set)
341     self.fft_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
342     scrollbar.config(command=self.fft_listbox.yview)
343
344     btn_frame = tk.Frame(control, bg=COLORS['card'])
345     btn_frame.pack(fill=tk.X, padx=10, pady=5)
346
347     tk.Button(btn_frame, text="          Tracer les FFT",
348             command=self.plot_fft_spectra,
349             bg=COLORS['success'], fg='white', font=('Segoe UI', 9,
350             'bold'),
351             relief=tk.FLAT, cursor='hand2', padx=10,
352             pady=4).pack(fill=tk.X, pady=2)
353     tk.Button(btn_frame, text="          Actualiser la liste",
354             command=self.update_fft_list,
355             bg=COLORS['secondary'], fg='white', font=('Segoe UI',
356             9),
357             relief=tk.FLAT, cursor='hand2', padx=10,
358             pady=4).pack(fill=tk.X, pady=2)
359
360     # Bouton sélectionner tout
361     tk.Button(btn_frame, text="          Sélectionner tout",
362             command=self.select_all_fft,
363             bg=COLORS['primary'], fg='white', font=('Segoe UI', 9),
364             relief=tk.FLAT, cursor='hand2', padx=10,
365             pady=4).pack(fill=tk.X, pady=2)
366
367     frame = tk.Frame(self.tab4, bg=COLORS['bg'])
368     frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
369
370     self.fft_fig, (self.fft_ax_dm, self.fft_ax_cm) =
371         plt.subplots(2, 1, figsize=(9, 7), facecolor='white')
372     self.fft_ax_dm.set_facecolor('#fafafa')

```

```

359 self.fft_ax_cm.set_facecolor('#fafafa')
360 self.fft_fig.suptitle('Analyse spectrale - Mode Différentiel
    (DM) et Mode Commun (CM)',
361                        fontsize=11, fontweight='bold')
362 self.fft_canvas = FigureCanvasTkAgg(self.fft_fig, master=frame)
363 self.fft_canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
364 toolbar = NavigationToolbar2Tk(self.fft_canvas, frame)
365
366 def create_table_tab(self):
367     main_frame = tk.Frame(self.tab5, bg=COLORS['bg'])
368     main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
369
370     info_frame = tk.Frame(main_frame, bg=COLORS['card'],
371                           relief=tk.RIDGE, bd=1)
372     info_frame.pack(fill=tk.X, pady=(0, 10))
373
374     self.table_info_label = tk.Label(info_frame, text="Aucun
375                                     fichier chargé",
376                                     bg=COLORS['card'],
377                                     fg=COLORS['danger'],
378                                     font=('Segoe UI', 10),
379                                     padx=10, pady=5)
380
381     self.table_info_label.pack(anchor=tk.W)
382
383     table_frame = tk.Frame(main_frame, bg=COLORS['bg'])
384     table_frame.pack(fill=tk.BOTH, expand=True)
385
386     v_scrollbar = tk.Scrollbar(table_frame, orient=tk.VERTICAL)
387     h_scrollbar = tk.Scrollbar(table_frame, orient=tk.HORIZONTAL)
388     v_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
389     h_scrollbar.pack(side=tk.BOTTOM, fill=tk.X)
390
391     self.tree = ttk.Treeview(table_frame,
392                             yscrollcommand=v_scrollbar.set,
393                             xscrollcommand=h_scrollbar.set,
394                             selectmode='extended')
395
396     v_scrollbar.config(command=self.tree.yview)
397     h_scrollbar.config(command=self.tree.xview)
398
399     self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
400
401     btn_frame = tk.Frame(main_frame, bg=COLORS['bg'])
402     btn_frame.pack(fill=tk.X, pady=(10, 0))
403
404     tk.Button(btn_frame, text="Charger Excel",
405               command=self.load_excel_file,
406               bg=COLORS['primary'], fg='white', font=('Segoe UI', 9),
407               relief=tk.FLAT, cursor='hand2', padx=10,
408               pady=4).pack(side=tk.LEFT, padx=5)
409
410     tk.Button(btn_frame, text="Rafraîchir",
411               command=self.refresh_table if self.data_loaded else None,
412               bg=COLORS['secondary'], fg='white', font=('Segoe UI',
413               9),
414               relief=tk.FLAT, cursor='hand2', padx=10,
415               pady=4).pack(side=tk.LEFT, padx=5)
416

```



```

447     rec_card = tk.Frame(frame, bg=COLORS['card'], relief=tk.RIDGE,
448         bd=1)
449     rec_card.pack(fill=tk.BOTH, expand=True)
450     scroll = tk.Scrollbar(rec_card)
451     scroll.pack(side=tk.RIGHT, fill=tk.Y)
452
453     self.recommend_text = tk.Text(rec_card, wrap=tk.WORD,
454         font=('Segoe UI', 10),
455         bg=COLORS['card'],
456         fg=COLORS['text'], padx=15,
457         pady=15,
458         yscrollcommand=scroll.set,
459         relief=tk.FLAT)
460     self.recommend_text.pack(side=tk.LEFT, fill=tk.BOTH,
461         expand=True)
462     scroll.config(command=self.recommend_text.yview)
463
464     btn_frame = tk.Frame(frame, bg=COLORS['bg'])
465     btn_frame.pack(fill=tk.X, pady=(10, 0))
466
467     tk.Button(btn_frame, text="Copier",
468         command=self.copy_recommendations,
469         bg=COLORS['secondary'], fg='white', padx=15, pady=4,
470         relief=tk.FLAT, cursor='hand2').pack(side=tk.LEFT,
471         padx=5)
472     tk.Button(btn_frame, text="Exporter rapport",
473         command=self.export_report if self.data_loaded else None,
474         bg=COLORS['primary'], fg='white', padx=15, pady=4,
475         relief=tk.FLAT, cursor='hand2').pack(side=tk.LEFT,
476         padx=5)
477
478     def create_statusbar(self):
479         self.status = tk.Frame(self.root, bg=COLORS['card'], height=28)
480         self.status.pack(side=tk.BOTTOM, fill=tk.X)
481
482         self.status_label = tk.Label(self.status, text="Pr t - Ouvrez
483             un fichier Excel pour commencer", bg=COLORS['card'],
484             fg=COLORS['text_light'],
485             font=('Segoe UI', 9))
486         self.status_label.pack(side=tk.LEFT, padx=15, pady=4)
487
488         self.progress = ttk.Progressbar(self.status, length=120,
489             mode='indeterminate')
490         self.progress.pack(side=tk.RIGHT, padx=15, pady=4)
491
492     #
493     =====
494     # FONCTIONS DE TABLEAU
495     #
496     =====
497
498     def clear_table(self):
499         for item in self.tree.get_children():
500             self.tree.delete(item)
501         self.tree['columns'] = []
502         self.table_info_label.config(text="Aucun fichier chargé",
503             fg=COLORS['danger'])

```



```

489
490 def refresh_table(self):
491     if self.df is None or len(self.df) == 0:
492         self.clear_table()
493         return
494
495     for item in self.tree.get_children():
496         self.tree.delete(item)
497
498     self.tree['columns'] = []
499     columns = list(self.df.columns)
500     self.tree['columns'] = columns
501
502     for col in columns:
503         width = 100 if col not in ['Fichier', 'Criticite'] else 150
504         self.tree.column(col, width=width, minwidth=80,
505             anchor='center')
506         self.tree.heading(col, text=col, anchor='center')
507
508     for idx, row in self.df.iterrows():
509         values = []
510         for col in columns:
511             val = row[col]
512             if isinstance(val, float):
513                 if col in ['DM_max (dB V)', 'CM_max (dB V)']:
514                     values.append(f"{val:.2f}")
515                 else:
516                     values.append(f"{val:.0f}" if val == int(val)
517                         else f"{val}")
518             else:
519                 values.append(str(val))
520         self.tree.insert('', tk.END, values=values)
521
522     if 'Criticite' in self.df.columns:
523         self.tree.tag_configure('critique', background='#ffcccc')
524         self.tree.tag_configure('eleve', background='#ffe0cc')
525         self.tree.tag_configure('modere', background='#fff0cc')
526
527     for idx, row in self.df.iterrows():
528         item = self.tree.get_children()[idx]
529         if row['Criticite'] == 'Critique':
530             self.tree.item(item, tags=('critique',))
531         elif row['Criticite'] == 'Élevé':
532             self.tree.item(item, tags=('eleve',))
533         elif row['Criticite'] == 'Modéré':
534             self.tree.item(item, tags=('modere',))
535
536     self.table_info_label.config(text=f"          {len(self.df)}
537     simulations | {os.path.basename(self.current_file) if
538     self.current_file else 'Données'}", fg=COLORS['success'])
539
540 def copy_table(self):
541     if self.df is None or len(self.df) == 0:
542         messagebox.showwarning("Attention", "Aucune donnée à
543         copier")
544         return
545
546     clipboard_text = self.df.to_string(index=False)

```

```

542     self.root.clipboard_clear()
543     self.root.clipboard_append(clipboard_text)
544     messagebox.showinfo("Succès", "Tableau copié!")
545
546     #
547     # CHARGEMENT DES DONNÉES
548     #
549     # =====
550
551     def load_excel_file(self):
552         filepath = filedialog.askopenfilename(
553             title="Sélectionner le fichier Excel",
554             filetypes=[("Excel files", "*.xlsx *.xls"), ("All files",
555                 " *.*")]
556         )
557         if filepath:
558             self.clear_table()
559             self.load_data_from_file(filepath)
560
561     def load_data_from_file(self, filepath):
562         try:
563             self.progress.start()
564             self.status_label.config(text="Chargement...")
565             self.root.update()
566
567             self.df = pd.read_excel(filepath)
568
569             print("Colonnes trouvées:", list(self.df.columns))
570
571             if 'DM_max (dB V)' not in self.df.columns and 'DM_max (V)'
572                 in self.df.columns:
573                 self.df['DM_max (dB V)'] = self.df['DM_max (V)']
574             if 'CM_max (dB V)' not in self.df.columns and 'CM_max (V)'
575                 in self.df.columns:
576                 self.df['CM_max (dB V)'] = self.df['CM_max (V)']
577
578             needed = ['Vin (V)', 'fsw (Hz)', 'Iload (A)', 'DM_max
579                 (dB V)', 'CM_max (dB V)']
580
581             for col in needed:
582                 if col not in self.df.columns:
583                     raise Exception(f"Colonne manquante: {col}")
584
585             for col in needed:
586                 self.df[col] = pd.to_numeric(self.df[col],
587                     errors='coerce')
588
589             self.df = self.df.dropna()
590
591             if len(self.df) == 0:
592                 raise Exception("Aucune donnée valide")
593
594             def get_critique(val):
595                 if val > 150:
596                     return 'Critique'
597                 elif val > 145:
598                     return 'Élevé'

```

```

593         elif val > 140:
594             return 'Modéré'
595         else:
596             return 'Faible'
597
598     self.df['Criticite'] = self.df['DM_max
        (dB V)'].apply(get_critique)
599
600     self.generate_fft_data()
601
602     self.data_loaded = True
603     self.current_file = filepath
604     self.file_label.config(text=os.path.basename(filepath),
        fg=COLORS['success'])
605     self.status_label.config(text=f"          Chargé: {len(self.df)}
        simulations")
606
607     self.update_all_filters()
608     self.update_graph_1()
609     self.update_graph_2()
610     self.update_graph_3()
611     self.update_heatmap()
612     self.refresh_table()
613     self.update_recommendations()
614     self.update_fft_list()
615     self.plot_critical_zones()
616
617     messagebox.showinfo("Succès",
618         f"Fichier chargé!\n\n"
619         f"          {len(self.df)} simulations\n"
620         f"      Vin: {self.df['Vin (V)'].min():.0f}-{self.df['Vin
        (V)'].max():.0f}V\n"
621         f"      fsw: {self.df['fsw
        (Hz)'].min()/1000:.0f}-{self.df['fsw
        (Hz)'].max()/1000:.0f}kHz\n"
622         f"      Iload: {self.df['Iload
        (A)'].min():.0f}-{self.df['Iload (A)'].max():.0f}A")
623
624     except Exception as e:
625         messagebox.showerror("Erreur", f"Erreur de
        chargement:\n\n{str(e)}")
626         self.status_label.config(text="          Erreur")
627         self.data_loaded = False
628         self.df = None
629         self.clear_table()
630         self.show_no_data_state()
631     finally:
632         self.progress.stop()
633
634     def update_all_filters(self):
635         if self.df is None:
636             return
637
638         vin_vals = ["Tous"] + sorted([f"{v:.0f}" for v in self.df['Vin
        (V)'].unique()])
639         fsw_vals = ["Tous"] + sorted([f"{int(v/1000)}" for v in
        self.df['fsw (Hz)'].unique()])

```

```

640     iload_vals = ["Tous"] + sorted([f"{v}" for v in self.df['Iload
        (A)'].unique()])
641
642     self.vin_combo_1['values'] = vin_vals
643     self.fsw_combo_1['values'] = fsw_vals
644     self.iloader_combo_1['values'] = iload_vals
645
646     self.vin_combo_2['values'] = vin_vals
647     self.fsw_combo_2['values'] = fsw_vals
648     self.iloader_combo_2['values'] = iload_vals
649
650     self.vin_combo_3['values'] = vin_vals
651     self.fsw_combo_3['values'] = fsw_vals
652     self.iloader_combo_3['values'] = iload_vals
653
654     def get_filtered_df_1(self):
655         if self.df is None:
656             return None
657         df_f = self.df.copy()
658         if self.vin_var_1.get() != "Tous":
659             df_f = df_f[df_f['Vin (V)'] == float(self.vin_var_1.get())]
660         if self.fsw_var_1.get() != "Tous":
661             df_f = df_f[df_f['fsw (Hz)'] == int(self.fsw_var_1.get()) *
                1000]
662         if self.iloader_var_1.get() != "Tous":
663             df_f = df_f[df_f['Iload (A)'] ==
                float(self.iloader_var_1.get())]
664         return df_f
665
666     def get_filtered_df_2(self):
667         if self.df is None:
668             return None
669         df_f = self.df.copy()
670         if self.vin_var_2.get() != "Tous":
671             df_f = df_f[df_f['Vin (V)'] == float(self.vin_var_2.get())]
672         if self.fsw_var_2.get() != "Tous":
673             df_f = df_f[df_f['fsw (Hz)'] == int(self.fsw_var_2.get()) *
                1000]
674         if self.iloader_var_2.get() != "Tous":
675             df_f = df_f[df_f['Iload (A)'] ==
                float(self.iloader_var_2.get())]
676         return df_f
677
678     def get_filtered_df_3(self):
679         if self.df is None:
680             return None
681         df_f = self.df.copy()
682         if self.vin_var_3.get() != "Tous":
683             df_f = df_f[df_f['Vin (V)'] == float(self.vin_var_3.get())]
684         if self.fsw_var_3.get() != "Tous":
685             df_f = df_f[df_f['fsw (Hz)'] == int(self.fsw_var_3.get()) *
                1000]
686         if self.iloader_var_3.get() != "Tous":
687             df_f = df_f[df_f['Iload (A)'] ==
                float(self.iloader_var_3.get())]
688         return df_f
689
690     def generate_fft_data(self):

```

```

691         if self.df is None:
692             return
693
694         self.fft_data = {}
695         fs = 10e6
696         t = np.linspace(0, 1e-3, 5000)
697
698         for idx, row in self.df.iterrows():
699             if pd.isna(row['DM_max (dB V)']):
700                 continue
701
702             key = f"Vin{row['Vin (V)']:.0f}_fsw{int(row['fsw (Hz)']/1000)}kHz_Iload{row['Iload (A)']}"
703             f0 = row['fsw (Hz)']
704
705             sig_dm = 0
706             for k in range(1, 9):
707                 sig_dm += (row['DM_max (dB V)'] / 60) * (1/k) *
708                     np.sin(2*np.pi*k*f0*t)
709
710             sig_cm = 0
711             for k in range(1, 9):
712                 sig_cm += (row['CM_max (dB V)'] / 60) * (1/k) *
713                     np.sin(2*np.pi*k*f0*t)
714
715             n = len(t)
716             freq = fftfreq(n, 1/fs)[:n//2]
717             fft_dm = np.abs(fft(sig_dm))[:n//2]
718             fft_cm = np.abs(fft(sig_cm))[:n//2]
719
720             self.fft_data[key] = {
721                 'freq': freq,
722                 'dm': 20*np.log10(fft_dm + 1e-10),
723                 'cm': 20*np.log10(fft_cm + 1e-10),
724                 'vin': row['Vin (V)'],
725                 'fsw': row['fsw (Hz)'],
726                 'iload': row['Iload (A)'],
727                 'criticite': row['Criticite']
728             }
729
730         #
731         =====
732         # GRAPHIQUES
733         #
734         =====
735
736         def update_graph_1(self):
737             if not self.data_loaded or self.df is None:
738                 return
739
740             df_f = self.get_filtered_df_1()
741             if df_f is None or len(df_f) == 0:
742                 self.ax1.clear()
743                 self.ax1.text(0.5, 0.5, 'Aucune donnée pour ces filtres',
744                             ha='center', va='center')
745                 self.canvas1.draw()
746                 return

```

```

743     self.ax1.clear()
744
745     vin_values = sorted(df_f['Vin (V)'].unique())
746     dm_means = []
747     dm_mins = []
748     dm_maxs = []
749     cm_means = []
750
751     for vin in vin_values:
752         subset = df_f[df_f['Vin (V)'] == vin]
753         dm_means.append(subset['DM_max (dB V)'].mean())
754         dm_mins.append(subset['DM_max (dB V)'].min())
755         dm_maxs.append(subset['DM_max (dB V)'].max())
756         cm_means.append(subset['CM_max (dB V)'].mean())
757
758     self.ax1.plot(vin_values, dm_means, 'o-', linewidth=2.5,
759                  markersize=8,
760                  color=COLORS['dm'], label='Mode Différentiel (DM)')
761     self.ax1.fill_between(vin_values, dm_mins, dm_maxs, alpha=0.2,
762                          color=COLORS['dm'])
763
764     self.ax1.plot(vin_values, cm_means, 's--', linewidth=2,
765                  markersize=7,
766                  color=COLORS['cm'], label='Mode Commun (CM)')
767
768     self.ax1.set_xlabel('Tension d\'entrée Vin (V)', fontsize=11)
769     self.ax1.set_ylabel('EMI max (dB V)', fontsize=11)
770
771     filters = []
772     if self.vin_var_1.get() != "Tous":
773         filters.append(f"Vin={self.vin_var_1.get()}V")
774     if self.fsw_var_1.get() != "Tous":
775         filters.append(f"fsw={self.fsw_var_1.get()}kHz")
776     if self.ilog_var_1.get() != "Tous":
777         filters.append(f"Iload={self.ilog_var_1.get()}A")
778
779     if filters:
780         title = f'Vin      EMI max ({", ".join(filters)})'
781     else:
782         title = 'Vin      EMI max (toutes configurations)'
783     self.ax1.set_title(title, fontsize=12, fontweight='bold')
784
785     self.ax1.legend(loc='best')
786     self.ax1.grid(True, alpha=0.3, linestyle='--')
787     self.ax1.axhspan(150, 200, alpha=0.15, color=COLORS['danger'])
788     self.ax1.axhspan(145, 150, alpha=0.15, color=COLORS['warning'])
789     self.ax1.axhspan(140, 145, alpha=0.15, color='#f1c40f')
790
791     self.canvas1.draw()
792
793     def update_graph_2(self):
794         if not self.data_loaded or self.df is None:
795             return
796
797         df_f = self.get_filtered_df_2()
798         if df_f is None or len(df_f) == 0:
799             self.ax2.clear()

```

```

797         self.ax2.text(0.5, 0.5, 'Aucune donnée pour ces filtres',
798                       ha='center', va='center')
799         self.canvas2.draw()
800         return
801
802     self.ax2.clear()
803
804     fsw_values = sorted(df_f['fsw (Hz)'].unique())
805     dm_means = []
806
807     for fsw in fsw_values:
808         subset = df_f[df_f['fsw (Hz)'] == fsw]
809         dm_means.append(subset['DM_max (dB V)'].mean())
810
811     self.ax2.plot([f/1000 for f in fsw_values], dm_means, 's-',
812                  linewidth=2.5, markersize=8, color=COLORS['primary'])
813
814     self.ax2.set_xlabel('Fréquence de commutation fsw (kHz)',
815                        fontsize=11)
816     self.ax2.set_ylabel('EMI max (dB V)', fontsize=11)
817
818     filters = []
819     if self.vin_var_2.get() != "Tous":
820         filters.append(f"Vin={self.vin_var_2.get()}V")
821     if self.fsw_var_2.get() != "Tous":
822         filters.append(f"fsw={self.fsw_var_2.get()}kHz")
823     if self.ilog_var_2.get() != "Tous":
824         filters.append(f"Iload={self.ilog_var_2.get()}A")
825
826     if filters:
827         title = f'fsw      Déplacement harmonique ({",
828                 ".join(filters)})'
829     else:
830         title = 'fsw      Déplacement harmonique (toutes
831                 configurations)'
832     self.ax2.set_title(title, fontsize=12, fontweight='bold')
833
834     self.ax2.grid(True, alpha=0.3, linestyle='--')
835     self.canvas2.draw()
836
837     def update_graph_3(self):
838         if not self.data_loaded or self.df is None:
839             return
840
841         df_f = self.get_filtered_df_3()
842         if df_f is None or len(df_f) == 0:
843             self.ax3.clear()
844             self.ax3.text(0.5, 0.5, 'Aucune donnée pour ces filtres',
845                           ha='center', va='center')
846             self.canvas3.draw()
847             return
848
849         self.ax3.clear()
850
851         iload_values = sorted(df_f['Iload (A)'].unique())
852         dm_means = []
853         cm_means = []

```

```

849     for iload in iload_values:
850         subset = df_f[df_f['Iload (A)'] == iload]
851         dm_means.append(subset['DM_max (dB V)'].mean())
852         cm_means.append(subset['CM_max (dB V)'].mean())
853
854     self.ax3.plot(iloading_values, dm_means, '^--', linewidth=2.5,
855                  markersize=8,
856                  color=COLORS['dm'], label='Mode Différentiel (DM)')
857     self.ax3.plot(iloading_values, cm_means, 'v--', linewidth=2,
858                  markersize=7,
859                  color=COLORS['cm'], label='Mode Commun (CM)')
860
861     self.ax3.set_xlabel('Courant de charge Iload (A)', fontsize=11)
862     self.ax3.set_ylabel('EMI max (dB V)', fontsize=11)
863
864     filters = []
865     if self.vin_var_3.get() != "Tous":
866         filters.append(f"Vin={self.vin_var_3.get()}V")
867     if self.fsw_var_3.get() != "Tous":
868         filters.append(f"fsw={self.fsw_var_3.get()}kHz")
869     if self.iloading_var_3.get() != "Tous":
870         filters.append(f"Iload={self.iloading_var_3.get()}A")
871
872     if filters:
873         title = f'Iload      EMI max ({", ".join(filters)})'
874     else:
875         title = 'Iload      EMI max (toutes configurations)'
876     self.ax3.set_title(title, fontsize=12, fontweight='bold')
877
878     self.ax3.legend(loc='best')
879     self.ax3.grid(True, alpha=0.3, linestyle='--')
880     self.canvas3.draw()
881
882     def update_heatmap(self):
883         if self.df is None:
884             return
885
886         col = 'DM_max (dB V)'
887         pivot = self.df.pivot_table(index='Vin (V)', columns='fsw
888                                     (Hz)', values=col, aggfunc='mean')
889
890         self.heatmap_ax.clear()
891         im = self.heatmap_ax.imshow(pivot.values, cmap='RdYlGn_r',
892                                    aspect='auto', origin='lower')
893
894         self.heatmap_ax.set_xticks(range(len(pivot.columns)))
895         self.heatmap_ax.set_xticklabels([f'{int(c/1000)}' for c in
896                                         pivot.columns])
897         self.heatmap_ax.set_yticks(range(len(pivot.index)))
898         self.heatmap_ax.set_yticklabels([f'{int(v)}' for v in
899                                         pivot.index])
900
901         self.heatmap_ax.set_xlabel('fréquence fsw (kHz)', fontsize=12)
902         self.heatmap_ax.set_ylabel('Vin (V)', fontsize=12)
903         self.heatmap_ax.set_title('Cartographie des émissions EMI (DM)
904                                   en fonction de Vin et fsw', fontsize=12, fontweight='bold')
905

```



```

899     if hasattr(self, 'heatmap_cbar') and self.heatmap_cbar is not
None:
900         try:
901             self.heatmap_cbar.remove()
902         except:
903             pass
904
905     self.heatmap_cbar = self.heatmap_fig.colorbar(im,
ax=self.heatmap_ax)
906     self.heatmap_cbar.set_label('EMI (dB V)', fontsize=10)
907
908     for i in range(len(pivot.index)):
909         for j in range(len(pivot.columns)):
910             val = pivot.values[i, j]
911             if not pd.isna(val):
912                 text_color = 'white' if val > 147 else 'black'
913                 self.heatmap_ax.text(j, i, f'{val:.0f}',
ha="center", va="center",
914                                     color=text_color, fontsize=9,
fontweight='bold')
915
916     self.heatmap_canvas.draw()
917
918     def select_all_fft(self):
919         """Sélectionne toutes les simulations dans la liste FFT"""
920         self.fft_listbox.select_set(0, tk.END)
921
922     def plot_fft_spectra(self):
923         if not self.data_loaded:
924             messagebox.showwarning("Attention", "Veuillez d'abord
charger un fichier Excel")
925             return
926
927         selected = self.fft_listbox.curselection()
928         if not selected:
929             messagebox.showwarning("Attention", "Sélectionnez au moins
une simulation")
930             return
931
932         keys = list(self.fft_data.keys())
933
934         self.fft_ax_dm.clear()
935         self.fft_ax_cm.clear()
936
937         colors = plt.cm.tab10(np.linspace(0, 1, len(selected)))
938
939         for idx, color in zip(selected, colors):
940             if idx >= len(keys):
941                 continue
942             key = keys[idx]
943             data = self.fft_data[key]
944             mask = (data['freq'] >= 10e3) & (data['freq'] <= 10e6)
945
946             label = f"Vin={data['vin']:.0f}V,
fsw={int(data['fsw']/1000)}kHz, Iload={data['iload']}A
[{'data['criticite']}]"
947

```

```

948         self.fft_ax_dm.semilogx(data['freq'][mask],
          data['dm'][mask],
          color=color, linewidth=1.5,
          label=label)
949
950     self.fft_ax_cm.semilogx(data['freq'][mask],
          data['cm'][mask],
          color=color, linewidth=1.5,
          label=label)
951
952
953     self.fft_ax_dm.set_xlim([10e3, 10e6])
954     self.fft_ax_dm.set_ylim([0, 180])
955     self.fft_ax_dm.set_ylabel('Amplitude (dB V)', fontsize=10)
956     self.fft_ax_dm.set_title('Mode Différentiel (DM) - Extraction
          FFT', fontsize=11, fontweight='bold')
957     self.fft_ax_dm.grid(True, alpha=0.3, which='both')
958     self.fft_ax_dm.legend(loc='upper right', fontsize=7)
959     self.fft_ax_dm.set_facecolor('#fafafa')
960
961     self.fft_ax_cm.set_xlim([10e3, 10e6])
962     self.fft_ax_cm.set_ylim([0, 180])
963     self.fft_ax_cm.set_xlabel('Fréquence (Hz)', fontsize=10)
964     self.fft_ax_cm.set_ylabel('Amplitude (dB V)', fontsize=10)
965     self.fft_ax_cm.set_title('Mode Commun (CM) - Extraction FFT',
          fontsize=11, fontweight='bold')
966     self.fft_ax_cm.grid(True, alpha=0.3, which='both')
967     self.fft_ax_cm.legend(loc='upper right', fontsize=7)
968     self.fft_ax_cm.set_facecolor('#fafafa')
969
970     self.fft_fig.tight_layout()
971     self.fft_canvas.draw()
972
973     def update_fft_list(self):
974         self.fft_listbox.delete(0, tk.END)
975         if self.fft_data:
976             for key in sorted(self.fft_data.keys()):
977                 self.fft_listbox.insert(tk.END, key)
978
979     def plot_critical_zones(self):
980         if self.df is None:
981             return
982
983         self.critical_ax.clear()
984
985         colors_map = {
986             'Critique': COLORS['danger'],
987             'Élevé': COLORS['warning'],
988             'Modéré': '#f1c40f',
989             'Faible': COLORS['success']
990         }
991
992         for level, color in colors_map.items():
993             subset = self.df[self.df['Criticite'] == level]
994             if len(subset) > 0:
995                 self.critical_ax.scatter(subset['Vin (V)'],
          subset['DM_max (dB V)'],
          s=120, c=color, alpha=0.7,
          label=level,
          edgecolors='black',

```

```

1097                                     linewidth=1)
1098
1099 self.critical_ax.set_xlabel('Vin (V)', fontsize=11)
1100 self.critical_ax.set_ylabel('EMI DM (dB V)', fontsize=11)
1101 self.critical_ax.set_title('Identification des zones de
fonctionnement critiques', fontsize=12, fontweight='bold')
1102 self.critical_ax.grid(True, alpha=0.3, linestyle='--')
1103 self.critical_ax.legend(loc='best', fontsize=10)
1104
1105 self.critical_ax.axhline(y=150, color=COLORS['danger'],
linestyle='--', linewidth=2)
1106 self.critical_ax.axhline(y=145, color=COLORS['warning'],
linestyle='--', linewidth=1.5)
1107 self.critical_ax.axhline(y=140, color='#f1c40f',
linestyle='--', linewidth=1)
1108
1109 self.critical_canvas.draw()
1110
1111 def update_recommendations(self):
1112     if self.df is None:
1113         return
1114
1115     self.recommend_text.delete(1.0, tk.END)
1116
1117     self.recommend_text.insert(tk.END, "="*55 + "\n")
1118     self.recommend_text.insert(tk.END, "          INTERPRÉTATION
PHYSIQUE\n")
1119     self.recommend_text.insert(tk.END, "="*55 + "\n\n")
1120
1121     corr_vin = self.df['Vin (V)'].corr(self.df['DM_max (dB V)'])
1122     corr_fsw = self.df['fsw (Hz)'].corr(self.df['DM_max (dB V)'])
1123     corr_oload = self.df['Iload (A)'].corr(self.df['DM_max
(dB V)'])
1124
1125     self.recommend_text.insert(tk.END, "          ANALYSE DES
CORRÉLATIONS\n")
1126     self.recommend_text.insert(tk.END, f"          Vin          EMI :
{corr_vin:.3f}\n")
1127     if corr_vin > 0.7:
1128         self.recommend_text.insert(tk.END, "          Forte
corrélation positive\n")
1129     self.recommend_text.insert(tk.END, f"          fsw          EMI :
{corr_fsw:.3f}\n")
1130     self.recommend_text.insert(tk.END, f"          Iload          EMI :
{corr_oload:.3f}\n\n")
1131
1132     critiques = self.df[self.df['Criticite'] == 'Critique']
1133     eleves = self.df[self.df['Criticite'] == 'Élevé']
1134
1135     if len(critiques) > 0:
1136         self.recommend_text.insert(tk.END, "          CONFIGURATIONS
CRITIQUES\n")
1137         for _, row in critiques.iterrows():
1138             self.recommend_text.insert(tk.END, f"
Vin={row['Vin (V)']:.0f}V, fsw={int(row['fsw
(Hz)']/1000)}kHz, Iload={row['Iload (A)']}A\n")
1139         self.recommend_text.insert(tk.END, "\n")

```

```

1040     if len(eleves) > 0:
1041         self.recommend_text.insert(tk.END, "          CONFIGURATIONS
          RISQUE ÉLEVÉ\n")
1042     for _, row in eleves.head(5).iterrows():
1043         self.recommend_text.insert(tk.END, f"
          Vin={row['Vin (V)']:.0f}V, fsw={int(row['fsw
          (Hz)']/1000)}kHz, Iload={row['Iload (A)']}A\n")
1044     self.recommend_text.insert(tk.END, "\n")
1045
1046     self.recommend_text.insert(tk.END, "          RECOMMANDATIONS
          TECHNIQUES\n")
1047     self.recommend_text.insert(tk.END, "      1. Utiliser la tension
          d'entrée la plus basse possible (12V ou 24V)\n")
1048     self.recommend_text.insert(tk.END, "      2. Augmenter fsw à
          150-200kHz pour déplacer les harmoniques\n")
1049     self.recommend_text.insert(tk.END, "      3. Ajouter un filtre
          d'entrée LC (fréquence coupure < fsw/10)\n")
1050     self.recommend_text.insert(tk.END, "      4. Utiliser des
          inductances de mode commun\n")
1051     self.recommend_text.insert(tk.END, "      5. Optimiser le layout
          PCB pour réduire les boucles de courant\n")
1052
1053     def copy_recommendations(self):
1054         self.recommend_text.clipboard_clear()
1055         self.recommend_text.clipboard_append(self.recommend_text.get(1.0,
          tk.END))
1056         messagebox.showinfo("Succès", "Texte copié!")
1057
1058     def export_report(self):
1059         if not self.data_loaded or self.df is None:
1060             messagebox.showwarning("Attention", "Aucune donnée à
          exporter")
1061             return
1062
1063         folder = filedialog.askdirectory(title="Dossier pour le
          rapport")
1064         if not folder:
1065             return
1066
1067         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
1068         report_path = f"{folder}/Rapport_EMI_{timestamp}.txt"
1069
1070         with open(report_path, 'w', encoding='utf-8') as f:
1071             f.write("="*55 + "\n")
1072             f.write("RAPPORT D'ANALYSE EMI - CONVERTISSEUR BUCK\n")
1073             f.write(f>Date: {datetime.now().strftime('%Y-%m-%d
          %H:%M:%S')}\n")
1074             f.write("="*55 + "\n\n")
1075             f.write(self.recommend_text.get(1.0, tk.END))
1076             f.write("\n\n" + "="*55 + "\n")
1077             f.write("DÉTAIL COMPLET DES SIMULATIONS\n")
1078             f.write("="*55 + "\n\n")
1079             f.write(self.df.to_string())
1080
1081         messagebox.showinfo("Succès", f"Rapport
          exporté:\n{report_path}")
1082
1083     def export_graphs(self):

```

```

1084     if not self.data_loaded:
1085         messagebox.showwarning("Attention", "Aucune donnée à
            exporter")
1086         return
1087
1088     folder = filedialog.askdirectory(title="Dossier pour les
            graphiques")
1089     if not folder:
1090         return
1091
1092     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
1093
1094     self.fig1.savefig(f"{folder}/01_Vin_vs_EMI_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1095     self.fig2.savefig(f"{folder}/02_fsw_vs_EMI_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1096     self.fig3.savefig(f"{folder}/03_Iload_vs_EMI_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1097     self.fft_fig.savefig(f"{folder}/04_FFT_Spectra_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1098     self.heatmap_fig.savefig(f"{folder}/05_Heatmap_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1099     self.critical_fig.savefig(f"{folder}/06_Critical_Zones_{timestamp}.png",
            dpi=150, bbox_inches='tight', facecolor='white')
1100
1101     messagebox.showinfo("Succès", f"6 graphiques exportés!")
1102
1103     def show_documentation(self):
1104         doc = """
1105         DOCUMENTATION - EMI ANALYZER
1106
1107         FONCTIONNALITÉS:
1108         Vin      EMI max : Évolution de l'EMI en fonction de Vin
1109         fsw      Déplacement harmonique : Impact de fsw
1110         Iload    EMI max : Effet du courant de charge
1111         FFT CM/DM : Analyse spectrale comparative avec sélection multiple
1112         Tableau Excel : Affichage des données chargées
1113         Cartographie EMI : Visualisation 2D Vin/fsw
1114         Zones critiques : Identification et recommandations
1115
1116         SEUILS DE CRITICITÉ:
1117         <140 dB V : Faible (Vert)
1118         140-145 dB V : Modéré (Jaune)
1119         145-150 dB V : Élevé (Orange)
1120         >150 dB V : Critique (Rouge)
1121
1122         RACCOURCIS CLAVIER:
1123         Ctrl+O : Ouvrir fichier Excel
1124         Ctrl+E : Exporter tous les graphiques
1125         Ctrl+Q : Quitter
1126         """
1127         messagebox.showinfo("Documentation", doc)
1128
1129     def show_about(self):
1130         about = """
1131         EMI ANALYZER - Convertisseur Buck
1132         Version 2.0
1133

```

```
1134 Développé pour l'étude paramétrique des
1135 perturbations électromagnétiques.
1136
1137 Fonctionnalités:
1138     Analyse Vin, fsw, Iload vs EMI
1139     Extraction FFT avec sélection multiple
1140     Cartographie interactive
1141     Affichage tableau Excel
1142     Identification zones critiques
1143     Export de rapports complets
1144
1145 Technologies:
1146 Python      Tkinter      Matplotlib      Pandas      NumPy      SciPy
1147
1148     2026 - Projet EMI Buck Converter
1149 """
1150     messagebox.showinfo("    propos", about)
1151
1152
1153 def main():
1154     root = tk.Tk()
1155     app = EMIAalyzerApp(root)
1156     root.mainloop()
1157
1158
1159 if __name__ == "__main__":
1160     main()
```

Listing C.1 – Code complet de EMIAalyzerApp